

Информационные технологии и безопасность
ПРОТОКОЛ ЗАЩИТЫ ТРАНСПОРТНОГО УРОВНЯ
ВЕРСИИ 1.3

Інфармацыйныя тэхналогіі і бяспека
ПРАТАКОЛ АХОВЫ ТРАНСПАРТНАГА ЎЗРОЎНЮ
ВЕРСІІ 1.3

(RFC 8446:2018, NEQ)



УДК 004.728.5.057.4.056.5(083.74) (476)

ОГКС 35.240.01

NEQ

Ключевые слова: транспортный уровень, криптографический протокол, шифрование, контроль целостности и подлинности, аутентификация

Предисловие

Цели, основные принципы, положения по государственному регулированию и управлению в области технического нормирования и стандартизации установлены Законом Республики Беларусь «О техническом нормировании и стандартизации».

1 РАЗРАБОТАН учреждением Белорусского государственного университета «Научно-исследовательский институт прикладных проблем математики и информатики»

2 УТВЕРЖДЕН И ВВЕДЕН В ДЕЙСТВИЕ постановлением Госстандарта Республики Беларусь от 27 ноября 2025 г. № 156

3 Настоящий стандарт разработан с учетом основных нормативных положений документа RFC 8446:2018 «The Transport Layer Security (TLS) Protocol. Version 1.3» («Протокол защиты транспортного уровня. Версия 1.3»)

4 ВВЕДЕН ВПЕРВЫЕ

Содержание

1	Область применения	1
2	Нормативные ссылки	1
3	Термины и определения	2
4	Сокращения, соглашения и обозначения	3
5	Общие положения	4
5.1	Назначение	4
5.2	Компоненты	4
5.3	Этапы Handshake	5
5.4	Перезапуск Handshake	6
5.5	Возобновление связи	7
5.6	Механизм 0-RTT	9
6	Протокол Handshake	10
6.1	Сообщения	10
6.2	Обмен ключами	11
6.3	Расширения	16
6.4	Параметры сервера	31
6.5	Аутентификация	32
6.6	Сообщение EndOfEarlyData	36
6.7	Сообщения после Handshake	36
7	Протокол Record	40
7.1	Записи	40
7.2	Фрагментация	40
7.3	Защита записей	42
7.4	Синхропосылки	43
7.5	Дополнение записей	43
7.6	Квоты ключей	44
8	Протокол Alert	44
8.1	Оповещения	44
8.2	Оповещения о закрытии соединения	45
8.3	Оповещения об ошибках	46
9	Криптографическая поддержка	47
9.1	Алгоритмы Hash и HMAC	47
9.2	Стенограммы	48
9.3	Алгоритмы построения ключей	48
9.4	Расписание ключей	50
9.5	Обновление ключей	51
9.6	Ключи защиты записей	52
9.7	Аутентификаторы	53
9.8	PSK -скрепки	54
9.9	Другие ключи и секреты	55
	Приложение А (обязательное) Правила описания данных	56
	Приложение Б (обязательное) Расширение extended_master_secret	60
	Приложение В (обязательное) Использование алгоритмов стандартов серии СТБ 34.101	62
	Библиография	65

ГОСУДАРСТВЕННЫЙ СТАНДАРТ РЕСПУБЛИКИ БЕЛАРУСЬ

**Информационные технологии и безопасность
ПРОТОКОЛ ЗАЩИТЫ ТРАНСПОРТНОГО УРОВНЯ ВЕРСИИ 1.3****Інфармацыйныя тэхналогіі і бяспека
ПРАТАКОЛ АХОВЫ ТРАНСПАРТНАГА ЎЗРОЎНЮ ВЕРСІІ 1.3****Information technology and security
The Transport Layer Security Protocol Version 1.3**

Дата введения 2026-07-01**1 Область применения**

Настоящий стандарт устанавливает протокол TLS (Transport Layer Security) версии 1.3, который обеспечивает защиту данных, передаваемых через транспортный коммуникационный уровень. Стандарт определяет действия сторон протокола и форматы сообщений, которыми стороны обмениваются.

Настоящий стандарт применяется при разработке средств криптографической защиты информации.

2 Нормативные ссылки

СТБ 34.101.19-2012 Информационные технологии и безопасность. Форматы сертификатов и списков отозванных сертификатов инфраструктуры открытых ключей

СТБ 34.101.26-2012 Информационная технология и безопасность. Онлайновый протокол проверки статуса сертификата (OCSP)

СТБ 34.101.31-2020 Информационные технологии и безопасность. Алгоритмы шифрования и контроля целостности

СТБ 34.101.45-2013 Информационные технологии и безопасность. Алгоритмы электронной цифровой подписи и транспорта ключа на основе эллиптических кривых

СТБ 34.101.47-2017 Информационные технологии и безопасность. Криптографические алгоритмы генерации псевдослучайных чисел

СТБ 34.101.65-2014 Информационные технологии и безопасность. Протокол защиты транспортного уровня (TLS)

СТБ 34.101.66-2014 Информационные технологии и безопасность. Протоколы формирования общего ключа на основе эллиптических кривых

СТБ 34.101.77-2020 Информационные технологии и безопасность. Криптографические алгоритмы на основе sponge-функции

СТБ 34.101.78-2019 Информационные технологии и безопасность. Профиль инфраструктуры открытых ключей

ГОСТ 34.973-91 (ИСО 8824-87) Информационная технология. Взаимосвязь открытых систем. Спецификация абстрактно-синтаксической нотации версии 1 (ASN.1)

ГОСТ 27463-87 Системы обработки информации. 7-битные кодированные наборы символов

Примечание — При пользовании настоящим стандартом целесообразно проверить действие ссылочных документов на официальном сайте Национального фонда технических нормативных правовых актов в глобальной компьютерной сети Интернет.

Если ссылочные документы заменены (изменены), то при пользовании настоящим стандартом следует руководствоваться действующими взамен документами. Если ссылочные доку-

менты отменены без замены, то положение, в котором дана ссылка на них, применяется в части, не затрагивающей эту ссылку.

3 Термины и определения

В настоящем стандарте применяют термины, установленные в СТБ 34.101.19, СТБ 34.101.31, СТБ 34.101.45, СТБ 34.101.47 и СТБ 34.101.78, а также следующие термины с соответствующими определениями:

3.1 аутентификация: Проверка подлинности стороны.

3.2 билет (ticket): Ссылка на секрет, согласованный в соединении TLS.

3.3 возобновление связи (resumption): Ускоренный вариант Handshake, в котором стороны используют секрет, согласованный в предыдущем соединении.

3.4 запись (record): Единица данных в протоколе Record.

3.5 клиент (client): Сторона, которая инициирует выполнение протокола TLS.

3.6 криптонабор (cipher suite): Алгоритмы аутентифицированного шифрования в связке с алгоритмом хэширования, которые используются для защиты пересылаемых данных, построения ключей, аутентификации.

3.7 одноразовый ключ (ephemeral key): Ключ, который создается, используется и уничтожается в течение одного сеанса протокола.

3.8 оповещение (alert): Сообщение протокола Alert.

3.9 параметры защиты (security parameters): Параметры, которые определяют правила защиты соединения.

Примечание — Параметрами защиты являются версия TLS, криптонабор, параметры протокола Диффи — Хеллмана, перечни алгоритмов электронной цифровой подписи и предварительно согласованных секретов, режим формирования общих ключей.

3.10 предварительно согласованный секрет; PSK (pre-shared secret): Секрет, согласованный сторонами до установки соединения: за пределами TLS или в предыдущем соединении.

3.11 прикладной протокол (application protocol): Протокол, который выполняется поверх протокола TLS.

3.12 прикладные данные (application data): Данные прикладных протоколов, которые пересылаются с помощью TLS.

3.13 протокол защиты транспортного уровня; протокол TLS (transport layer security protocol): Определяемый в настоящем стандарте криптографический протокол, который обеспечивает аутентификацию сторон, конфиденциальность и контроль целостности и подлинности данных, передаваемых между сторонами на транспортном коммуникационном уровне.

3.14 протокол Диффи — Хеллмана: Криптографический протокол, с помощью которого две стороны формируют общий секретный ключ, используя циклическую группу, обмениваясь открытыми ключами, элементами этой группы, и держа в секрете соответствующие личные ключи.

Примечание 1 — Имеется несколько вариантов протокола в зависимости от того, какие ключи использует та или иная сторона: долговременные (статические) или одноразовые (эфемерные). В настоящем стандарте у обеих сторон протокола Диффи — Хеллмана ключи одноразовые.

Примечание 2 — В протоколе Диффи — Хеллмана используется конечная циклическая группа большого (как правило, простого) порядка. Обычно это либо подгруппа мультипликативной группы конечного поля, либо группа точек эллиптической кривой. Редакции протокола,

в которых используются группы первого типа, принято снабжать аббревиатурой FF (finite field), второго — аббревиатурой EC (elliptic curve). В настоящем стандарте группы различных типов не дифференцируются и аббревиатуры не используются.

3.15 протокол Alert: Протокол обмена сообщениями о закрытии соединения и об ошибках; часть протокола TLS.

3.16 протокол Handshake: Протокол согласования параметров, формирования общих ключей и аутентификации; часть протокола TLS.

3.17 протокол Record: Протокол передачи данных, в том числе в защищенном виде; часть протокола TLS.

3.18 расширение (extension): Структура данных, которая входит в сообщение Handshake и служит для согласования параметров, обмена ключевым материалом, аутентификации.

3.19 сервер (server): Сторона, которая выполняет протокол TLS по запросу клиента.

3.20 соединение (connection): Связь между сторонами на транспортном коммуникационном уровне.

3.21 стенограмма (протокола) (transcript): Последовательность сообщений протокола, переданных между сторонами к определенному моменту.

3.22 циклическая группа: Алгебраическая группа, все элементы которой являются степенями или кратными одного элемента.

Примечание — Элементы являются степенями при мультипликативной записи группы и кратными при аддитивной записи.

3.23 PSK-скрепка (PSK-binder): Данные, которые связывают предварительно согласованный секрет с текущим соединением и возможно одним из предыдущих.

4 Сокращения, соглашения и обозначения

В настоящем стандарте применяют следующие сокращения:

АСН.1 — абстрактно-синтаксическая нотация версии 1 (см. ГОСТ 34.973);

УЦ — удостоверяющий центр (см. СТБ 34.101.19, СТБ 34.101.78);

ЭЦП — электронная цифровая подпись (см. СТБ 34.101.45);

DER (distinguished encoding rules) — отличительные правила кодирования АСН.1 (см. СТБ 34.101.19 [приложение Б]);

DHE (Diffie-Hellman ephemeral) — протокол Диффи — Хеллмана с одноразовыми ключами;

OCSP (online certificate status protocol) — онлайн-протокол проверки статуса сертификата (см. СТБ 34.101.26);

RTT (round-trip time) — цикл приема-передачи данных в компьютерных сетях.

В настоящем стандарте для описания форматов данных используются соглашения и обозначения, определенные в приложении А. Применяемый синтаксис основан на языке программирования Си [1] и правилах, заданных в [2].

Для описания преобразований двоичных слов используются обозначения, введенные в СТБ 34.101.31. Октеты двоичного слова могут кодироваться графическими символами базовой таблицы КОИ-7 (ASCII), определенной в ГОСТ 27463 и представленной в СТБ 34.101.47. Полученная в результате кодирования строка символов окаймляется двойными кавычками. Например, строка "tls13" является кодовым представлением двоичного слова 746C73313320₁₆. Двойные кавычки внутри строк не допускаются. Пустая строка "" соответствует пустому слову ⊥.

В настоящем стандарте термины «октет» и «байт» взаимозаменяемы.

5 Общие положения

5.1 Назначение

Настоящий стандарт определяет криптографический протокол TLS версии 1.3 (далее — TLS). Протокол предназначен для защиты соединений между клиентами и серверами в сети Интернет. Протокол определяется в соответствии со спецификацией [3]. Действия сторон протокола и форматы пересылаемых между сторонами сообщений представлены с такой степенью детализации, которая позволяет разрабатывать полностью совместимые между собой реализации TLS.

TLS встраивается в стек коммуникационных протоколов поверх транспортного уровня и обеспечивает защиту данных, передаваемых через этот уровень прикладными протоколами клиента и сервера. TLS выполняется независимо от прикладных протоколов и прозрачен для них.

С помощью TLS клиент и сервер создают защищенное соединение, в рамках которого проводится аутентификация сторон, обеспечиваются конфиденциальность и контроль целостности и подлинности передаваемых данных. Для организации соединения достаточно обеспечить надежную передачу данных на транспортном уровне.

Протокол TLS предыдущей версии 1.2 установлен в СТБ 34.101.65. Настоящий стандарт не отменяет СТБ 34.101.65, оба стандарта, как и соответствующие версии TLS, могут применяться одновременно независимо друг от друга.

Настоящий стандарт уточняет СТБ 34.101.65 в двух аспектах. Во-первых, в приложении Б определяется обязательное к применению расширение, которое защищает от атаки, обнаруженной после введения СТБ 34.101.65 в действие. Во-вторых, в необходимых случаях детализируется взаимодействие сторон TLS 1.2 с клиентами и серверами, которые поддерживают версию 1.3 протокола.

5.2 Компоненты

TLS состоит из трех субпротоколов: Handshake, Record и Alert (см. разделы 6 — 8).

Протокол Handshake позволяет клиенту и серверу аутентифицировать друг друга, согласовать параметры, в том числе версию TLS и перечень криптографических алгоритмов, сформировать общие ключи. Аутентификация сервера в Handshake обязательна, клиента — нет. Протокол противостоит вмешательству со стороны противника. В частности, противник не может навязать сторонам параметры, отличные от тех, которые были бы согласованы при отсутствии вмешательства.

В Handshake предусмотрены три режима формирования общих ключей:

- 1) DHE — протокол Диффи – Хеллмана с одноразовыми ключами;
- 2) PSK — на основе предварительно согласованных секретов;
- 3) PSK+DHE — составной режим, композиция двух предыдущих.

Параметры и ключевой материал режимов передается в специальных расширениях сообщений Handshake. В расширениях могут передаваться другие параметры и настройки TLS. Перечень расширений может пополняться.

Аутентификация основана на знании личных ключей, связанных с сертификатами открытых ключей, и (или) на знании предварительно согласованных секретов. Используются сертификаты, содержание и формат которых определены в СТБ 34.101.19.

Протокол Record использует согласованные при выполнении Handshake параметры и сформированные общие ключи для защиты данных обмена между клиентом и сервером.

Данные разбиваются на фрагменты, которые независимо обрабатываются на ключах защиты.

Протокол Alert извещает о закрытии соединений и об ошибках в процессе выполнения Handshake и Record.

Защита данных в протоколе Record выполняется с помощью алгоритмов аутентифицированного шифрования. Их ключи формируются с помощью алгоритмов схемы HKDF (HMAC-based Key Derivation Function), введенной в [4], [5]. Алгоритмы HKDF, правила формирования ключей с их помощью, другие криптографические аспекты TLS описаны в разделе 9.

В HKDF используются алгоритмы имитозащиты схемы HMAC, установленной в СТБ 34.101.47. Параметром HMAC и, следовательно, HKDF является алгоритм хэширования. Этот алгоритм в связке с алгоритмами аутентифицированного шифрования образует криптонабор TLS. В TLS предусмотрена возможность расширения перечня криптонаборов. В приложении В определены криптонаборы, основанные на алгоритмах, введенных в стандартах серии СТБ 34.101.

Криптонабор определяет правила криптографических вычислений за пределами Record. Во-первых, сообщения Handshake и Alert могут защищаться по той же схеме, что и фрагменты Record. Во-вторых, алгоритм имитозащиты HMAC, определяемый алгоритмом хэширования криптонабора, может использоваться в Handshake самостоятельно, не в качестве компонента HKDF.

Клиент и сервер согласуют используемый криптонабор в начале выполнения Handshake. Кроме этого, стороны информируют друг друга о своих криптографических возможностях: поддерживаемых циклических группах для протокола Диффи — Хеллмана, поддерживаемых алгоритмах ЭЦП для аутентификации по сертификату и для проверки сертификата.

5.3 Этапы Handshake

Сеанс Handshake состоит из следующих этапов.

1 Обмен ключами. Стороны согласуют параметры защиты и обмениваются ключевым материалом. По результатам обмена стороны формируют общие ключи, которые используются для защиты всех последующих сообщений.

2 Параметры сервера. Сервер устанавливает дополнительные параметры. Например, запрашивает аутентификацию клиента или настраивает поддержку определенных прикладных протоколов.

3 Аутентификация. Проводится аутентификация сервера и, возможно, клиента. Аутентификация сопровождается подтверждением корректности общих ключей и целостности сообщений Handshake.

На первом этапе клиент отправляет серверу приветственное сообщение **ClientHello**. Сообщение содержит случайные данные клиента, перечень поддерживаемых им версий TLS, список поддерживаемых криптонаборов, открытые ключи протокола Диффи — Хеллмана для режима DHE и (или) набор идентификаторов секретов для режима PSK, другие данные. Ключи и параметры для режимов DHE и PSK передаются в расширениях **key_share** и **pre_shared_key** соответственно.

Сервер обрабатывает **ClientHello**, выбирает параметры защиты и отвечает собственным приветственным сообщением **ServerHello**, в котором указывает эти параметры. При использовании режима DHE сообщение **ServerHello** содержит расширение **key_share** с открытым ключом Диффи — Хеллмана, сгенерированным сервером, а при использовании режима PSK — расширение **pre_shared_key** с номером секрета, выбранного сервером. При

использовании режима PSK+DHE сообщение **ServerHello** содержит оба расширения. Как и клиент, сервер включает в свое приветственное сообщение случайные данные. Наличие случайных данных обеспечивает общую волатильность сообщений Handshake даже при повторе всех остальных их частей.

На втором этапе сервер отправляет сообщение **EncryptedExtensions**. В него включаются расширения, которые не относятся к согласованию параметров защиты (и поэтому могут быть защищены) и не касаются сертификатов для аутентификации.

Если предусмотрена аутентификация клиента по сертификату, то сервер дополнительно отправляет сообщение **CertificateRequest**.

На третьем этапе клиент и сервер обмениваются сообщениями, которые прямо или косвенно поддерживают аутентификацию.

Во-первых, в сообщениях **Certificate** стороны пересылают друг другу свои сертификаты. Сервер не высылает сообщение **Certificate**, если решено использовать PSK, а клиент — если сервер не запрашивал аутентификацию на основе сертификата, т. е. не высылал сообщение **CertificateRequest**.

Во-вторых, в сообщениях **CertificateVerify** стороны пересылают свои подписи стенограмм Handshake. Подпись проверяется на открытом ключе из сертификата стороны-подписанта. Корректность подписи доказывает владение соответствующим личным ключом и, таким образом, подлинность стороны. Сообщение **CertificateVerify** не высылается, если аутентификация стороны по ее сертификату не предусмотрена.

В-третьих, в сообщении **Finished** стороны обмениваются имитовставками стенограмм Handshake. Имитовставки вычисляются на общих ключах, сформированных на этапе 1. Корректность имитовставки подтверждает знание общих ключей и, как следствие, владение стороной ключами, описанными в **ClientHello** или **ServerHello**. При использовании режима PSK подтверждается владение предварительно согласованным секретом и, таким образом, подлинность стороны.

Этапы Handshake представлены на рисунке 1. Здесь и на следующих рисунках знак «+» указывает на требующие внимания расширения, знак «*» — на необязательные сообщения или расширения. Фигурными скобками окаймляются сообщения, защищенные на общих ключах для Handshake, квадратными — на ключах для Record (см. 9.4).

Отправка сообщения **Finished** означает, что сторона завершила Handshake и может переходить к отправке данных прикладных протоколов («**Application Data**» на рисунке). Для ускорения работы сервер может отправлять прикладные данные, даже не получив встречного сообщения **Finished** от клиента. При планировании использования TLS в прикладных протоколах следует учитывать, что в момент отправки сервер взаимодействует с еще неаутентифицированным клиентом.

Еще одним способом ускорения TLS является отправка клиентом прикладных данных до завершения Handshake, т. е. до отправки собственного сообщения **Finished**. Этот способ описан в 5.6.

В TLS предусмотрена возможность отправки некоторых сообщений после Handshake. С их помощью выпускаются билеты, выполняется отложенная аутентификация, стороны информируют друг друга об обновлении ключей. Сообщения после Handshake защищаются так же, как прикладные данные.

5.4 Перезапуск Handshake

Содержание расширения **key_share**, передаваемого клиентом в **ClientHello**, может не устроить сервер. Для ускорения работы сервер вместо отмены Handshake может продолжить его выполнять при условии корректировки расширения. Для этого сервер от-

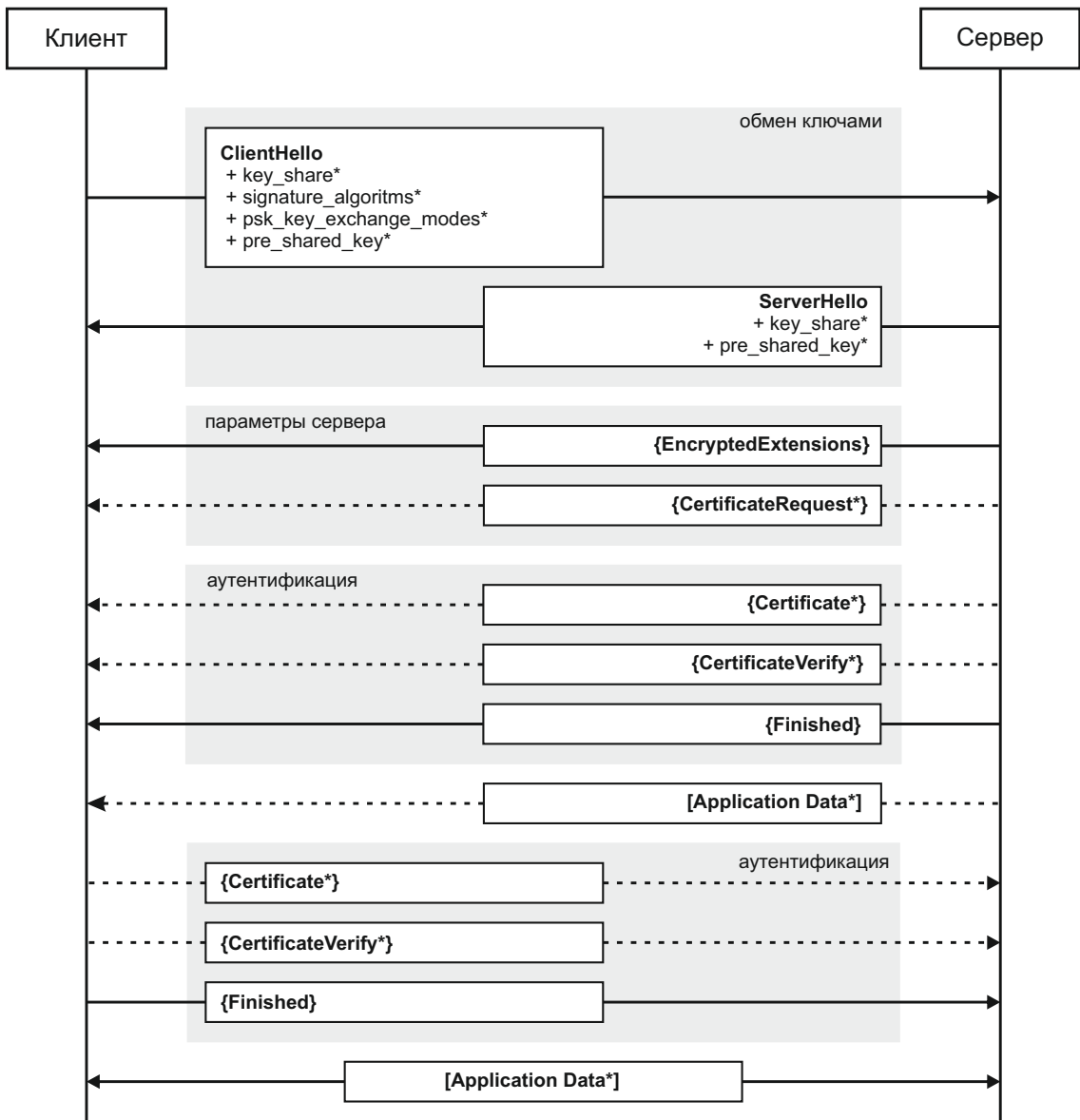


Рисунок 1 — Этапы Handshake

правляет клиенту сообщение **HelloRetryRequest**, которое включает `key_share` с описанием корректировок. Если клиент согласен с корректировками, то он высылает серверу повторное сообщение **ClientHello** с подправленным расширением, перезапуская таким способом Handshake (см. рисунок 2).

Пусть, например, в `key_share` первого **ClientHello** перечислены циклические группы режима DHE, ни одна из которых не устраивает сервер. Тогда сервер указывает допустимую группу в **HelloRetryRequest** и клиент использует ее в повторном **ClientHello**.

При перезапуске Handshake могут выполняться другие корректировки **ClientHello** (см. 6.2.2).

При перезапуске первое сообщение **ClientHello** и ответное **HelloRetryRequest** остаются в стенограмме Handshake.

5.5 Возобновление связи

Секрет режима PSK может быть согласован как за пределами TLS, так и в самом протоколе. В последнем случае секрет является одним из общих ключей, формируемых

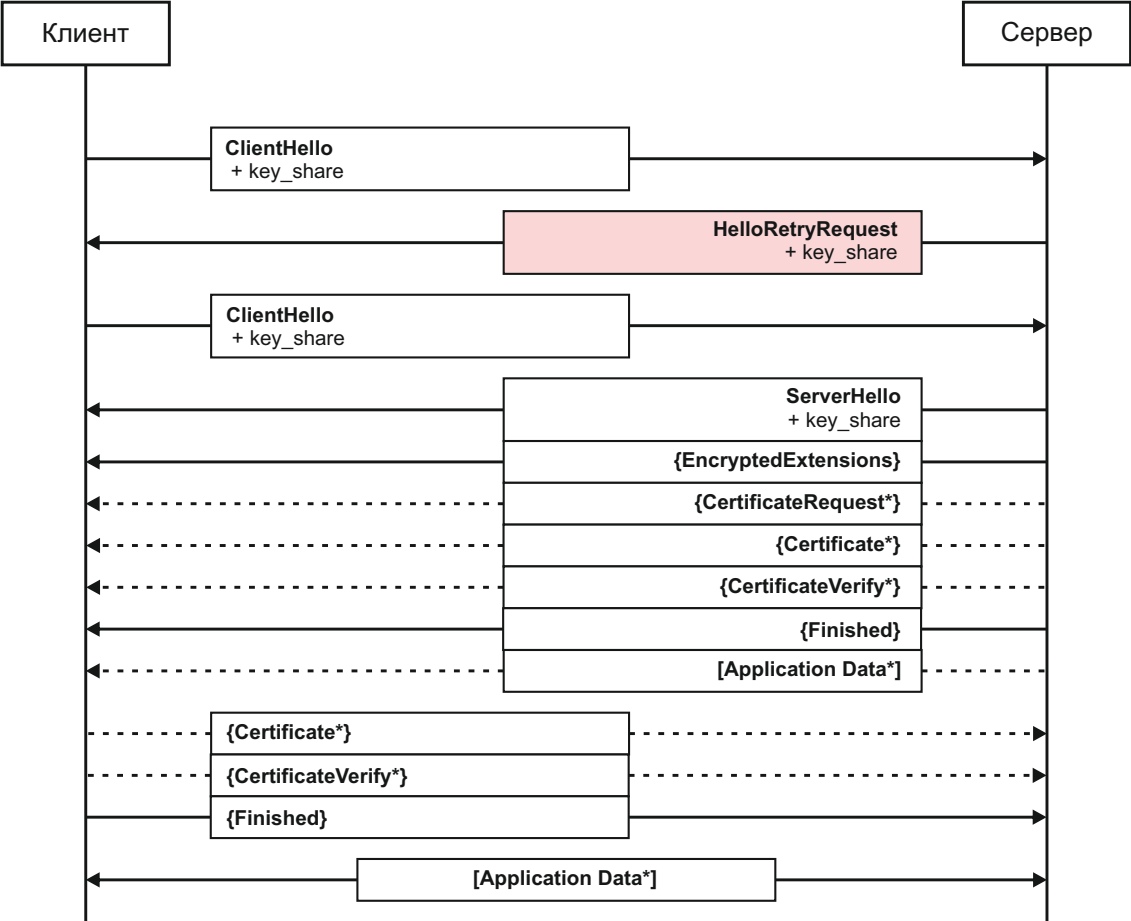


Рисунок 2 — Перезапуск Handshake

в сеансе Handshake. Клиент и сервер используют секрет в следующих сеансах протокола, возобновляя уже налаженную защищенную связь друг с другом. При возобновлении связи стороны используют ранее согласованные параметры и ключи, ускоряя выполнение Handshake.

Для возобновления связи сервер по завершении Handshake направляет клиенту сообщение **NewSessionTicket**, в котором указывает идентификатор сформированного секрета. Клиент сохраняет секрет вместе с присланным идентификатором. Клиент может хранить несколько секретов, полученных по завершении различных сеансов Handshake. В **ClientHello** нового сеанса клиент включает расширения **psk_key_exchange_modes** и **pre_shared_key**. В первом расширении клиент указывает допустимые для него режимы формирования общих ключей, во втором — идентификаторы сохраненных секретов. Допустимыми являются режимы PSK и PSK+DHE. Последний обеспечивает защиту от «чтения назад», т. е. конфиденциальность переданных прикладных данных даже в случае компрометации секрета.

Пример возобновления связи представлен на рисунке 3. В первом сеансе Handshake стороны формируют общий секрет, во втором — используют его. Во втором сеансе общие ключи формируются в режиме PSK и поэтому сервер не отправляет клиенту ни свой сертификат, ни сообщение **CertificateVerify**. Тем не менее в сообщении **ClientHello** второго сеанса клиент может выслать расширение **key_share**, допуская, что сервер откажется от возобновления связи и будет выполнять полный Handshake.

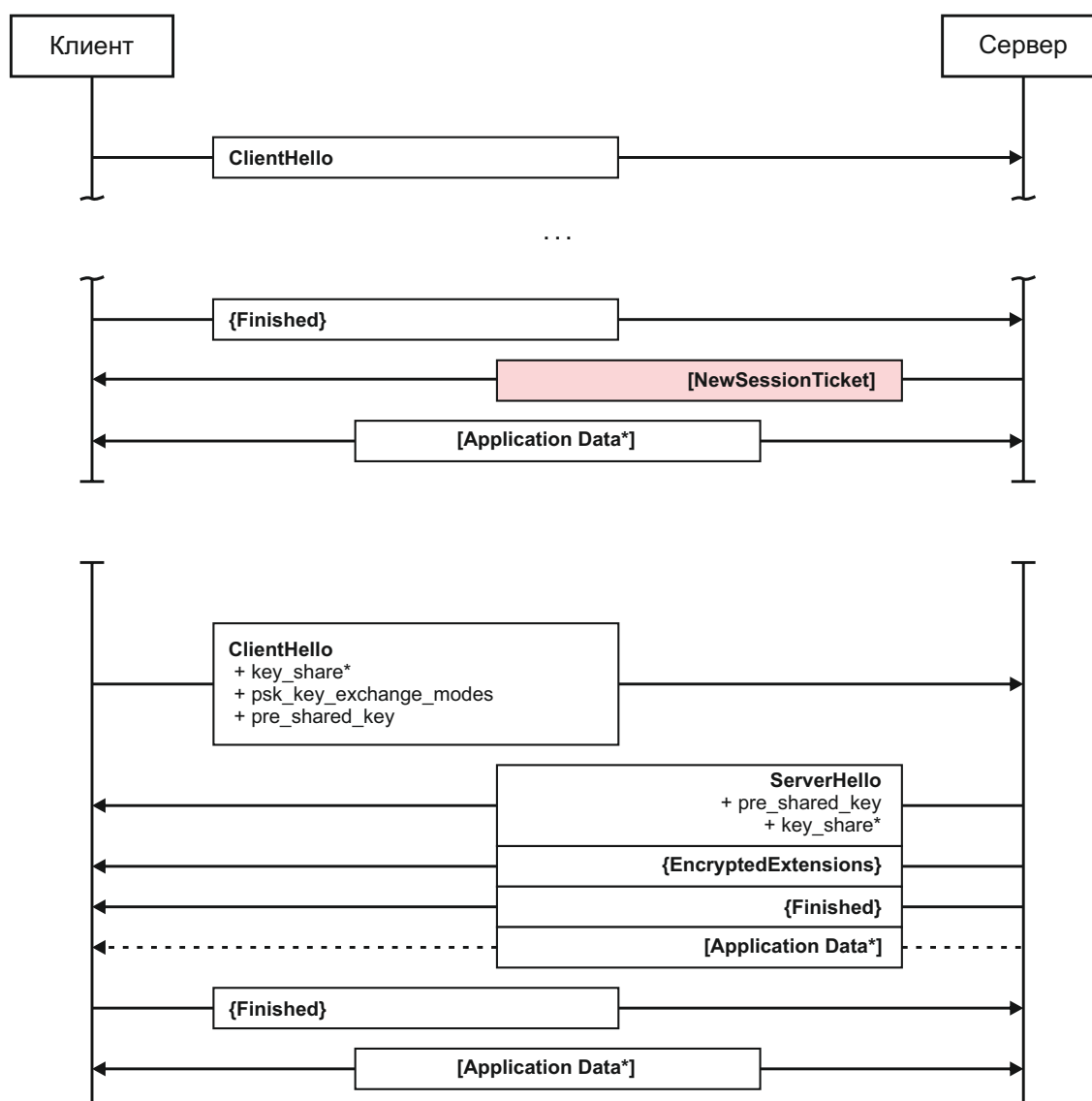


Рисунок 3 — Возобновление связи

5.6 Механизм 0-RTT

Располагая согласованным с сервером секретом режима PSK, клиент может начать пересылку прикладных данных сразу после отправки сообщения **ClientHello**, не дожидаясь ответа сервера. Данные защищаются на общем ключе, который строится по общему секрету. Ранняя отправка прикладных данных известна как механизм 0-RTT. Для сравнения, отправка данных клиентом после собственного сообщения **Finished** без перезапуска Handshake классифицируется как 1-RTT, с перезапуском — как 2-RTT.

Клиент сигнализирует о применении механизма 0-RTT, включая в **ClientHello** специальное расширение **early_data**. В расширении **pre_shared_key** клиент пересылает список идентификаторов предварительно согласованных секретов. Ключ механизма 0-RTT строится по первому секрету этого списка. Сервер объявляет о поддержке механизма, включая расширение **early_data** в свое сообщение **ServerHello**.

Пример применения механизма 0-RTT представлен на рисунке 4. Круглые скобки означают защиту ранних прикладных данных.

При планировании использования TLS в прикладных протоколах следует учитывать, что данные 0-RTT не защищены от «чтения назад» и что не обеспечена защита от повтора

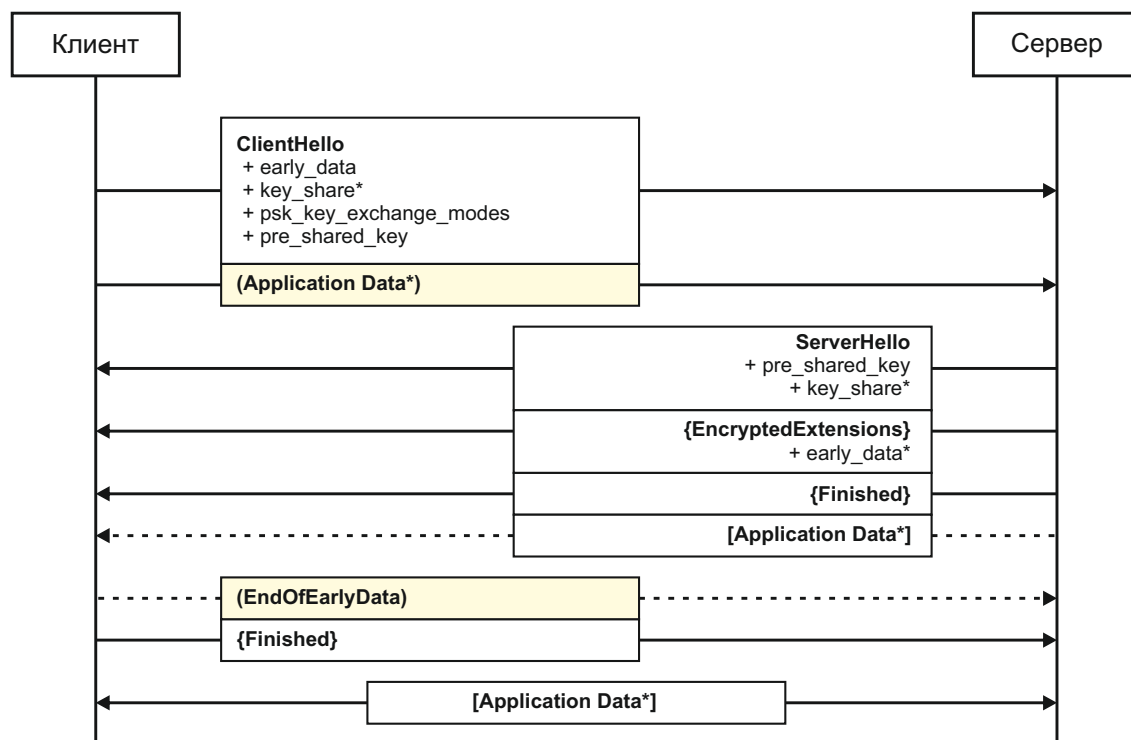


Рисунок 4 — Механизм 0-RTT

этих данных в других соединениях с сервером (эти соединения может открывать противник без ведома клиента). С другой стороны, данные 0-RTT не могут быть продублированы в текущем соединении. Более того, противник не может выдать данные 0-RTT за данные 1-RTT или 2-RTT, поскольку для защиты используются разные ключи.

6 Протокол Handshake

6.1 Сообщения

Сообщения Handshake описываются одноименным типом, который определяется следующим образом:

```

enum {
    client_hello(1),
    server_hello(2),
    new_session_ticket(4),
    end_of_early_data(5),
    encrypted_extensions(8),
    certificate(11),
    certificate_request(13),
    certificate_verify(15),
    finished(20),
    key_update(24),
    message_hash(254),
    (255)
} HandshakeType;

struct {
    HandshakeType msg_type;

```

```

uint24 length;
select (Handshake.msg_type) {
    case client_hello:      ClientHello;
    case server_hello:      ServerHello;
    case end_of_early_data:  EndOfEarlyData;
    case encrypted_extensions: EncryptedExtensions;
    case certificate_request: CertificateRequest;
    case certificate:        Certificate;
    case certificate_verify: CertificateVerify;
    case finished:           Finished;
    case new_session_ticket: NewSessionTicket;
    case key_update:         KeyUpdate;
};
} Handshake;

```

Поля `Handshake` имеют следующее значение:

- `msg_type` — тип сообщения. Описывается перечислением `HandshakeType`;
- `length` — длина в байтах следующего поля.

В поле, которое следует за `length`, задается тело сообщения. Его формат определяется типом `msg_type`.

Сообщения `Handshake` передаются с помощью протокола `Record`. В зависимости от типа сообщения оно может передаваться в открытом виде или быть защищено.

Сообщения `Handshake` должны отправляться в порядке, который указан в 9.2 и представлен на рисунках 1 — 4. Сторона, которая получает сообщение с нарушением порядка, должна прервать `Handshake` с оповещением `unexpected_message`.

6.2 Обмен ключами

6.2.1 Согласование параметров

На этапе обмена ключами стороны согласуют параметры защиты. Согласование начинается клиент, высылая в своем сообщении `ClientHello` устраивающие его параметры.

Имеются четыре перечня параметров:

- 1 Перечень поддерживаемых криптонаборов.
- 2 Перечень поддерживаемых циклических групп для протокола Диффи — Хеллмана. Задается в расширении `supported_groups` (см. 6.3.9). Сопровождается расширением `key_share` (см. 6.3.10), в котором клиент указывает открытые ключи для некоторых или всех перечисленных групп.
- 3 Перечень поддерживаемых алгоритмов ЭЦП. Задается в расширении `signature_algorithms` (см. 6.3.4). Может сопровождаться расширением `signature_algorithms_cert` с перечнем поддерживаемых алгоритмов проверки подписей сертификатов.
- 4 Список идентификаторов предварительно согласованных секретов. Задается в расширении `pre_shared_key` (см. 6.3.13). Сопровождается расширением `psk_key_exchange_modes` (см. 6.3.11), где клиент перечисляет допустимые режимы формирования общих ключей с участием секретов.

Если сервер решает не использовать предварительно согласованные секреты, то он может делать выбор в первых трех перечнях независимо друг от друга. А именно сервер выбирает подходящий криптонабор, циклическую группу, алгоритмы ЭЦП. Одновременно с выбором алгоритмов ЭЦП сервер выбирает сертификат для аутентификации. Если

группы, предложенные клиентом в `supported_groups`, не устраивают сервер, то он должен закрыть соединение с оповещением `handshake_failure` или `insufficient_security`.

Если сервер решает использовать предварительно согласованный секрет, то он, кроме всего прочего, должен выбрать режим формирования общих ключей из списка, предложенного клиентом в `psk_key_exchange_modes`. Если сервер выбирает режим PSK, то отсутствие подходящих групп в `supported_groups` не является критическим.

Если сервер выбрал группу, для которой клиент не предъявил в `key_share` открытый ключ, то сервер должен выслать сообщение `HelloRetryRequest`.

Если сервер выбрал параметры, не отправляя `HelloRetryRequest`, то он указывает их в ответном сообщении `ServerHello`. Сообщение формируется по следующим правилам:

- если сервер решил использовать предварительно согласованный секрет, то он включает в `ServerHello` расширение `pre_shared_key` с номером секрета в списке клиента;
- если сервер решил использовать протокол Диффи — Хеллмана, то он включает в `ServerHello` расширение `key_share` с выбранной циклической группой и открытым ключом для нее.

Если сервер решил не использовать PSK, то он должен включить `key_share` в `ServerHello` и впоследствии должен аутентифицироваться по сертификату, высылая сообщения `Certificate` и `CertificateVerify` (см. 6.5). Если сервер решил использовать PSK, то он не должен аутентифицироваться по сертификату.

Если сервер не может поддержать предложенные клиентом параметры, то он должен прервать Handshake с оповещением `handshake_failure` или `insufficient_security`.

6.2.2 Сообщение ClientHello

Сообщение `ClientHello` является первым сообщением клиента, с его отправки начинается выполнение TLS.

Формат сообщения `ClientHello` описывается одноименным типом:

```
uint16 ProtocolVersion;
opaque Random[32];

uint8 CipherSuite[2];

struct {
    ProtocolVersion legacy_version = 0x0303; /* TLS v1.2 */
    Random random;
    opaque legacy_session_id<0..32>;
    CipherSuite cipher_suites<2..2^16-2>;
    opaque legacy_compression_methods<1..2^8-1>;
    Extension extensions<8..2^16-1>;
} ClientHello;
```

Поля `ClientHello` имеют следующее значение:

- `legacy_version` — устаревшее поле для номера версии TLS. По соображениям совместимости в поле должно быть установлено значение `0x0303`, которое представляет версию 1.2. Согласование версии выполняется с помощью расширения `supported_versions` (см. 6.3.2). Сервер должен проверить, что в поле установлено значение `0x0303`. При нарушении условия сервер должен закрыть соединение с оповещением `protocol_version`;
- `random` — случайные данные клиента;

- `legacy_session_id` — устаревшее поле для идентификатора сеанса TLS. В TLS версии 1.2 поле использовалось для возобновления связи. Если клиент получил идентификатор сеанса от сервера TLS версии 1.2 и желает возобновить с сервером связь, то он должен указать идентификатор в поле `legacy_session_id`. В остальных случаях поле должны быть пустым (иметь нулевую длину);

- `cipher_suites` — перечень криптонаборов, которые поддерживает клиент, в порядке убывания предпочтения. Криптонабор описывает алгоритмы аутентифицированного шифрования для защиты записей протокола Record и алгоритм хэширования для построения ключей и других криптографических операций. Предварительно согласованные секреты связываются с алгоритмами хэширования.

Если клиент в расширении `pre_shared_key` предлагает использовать предварительно согласованный секрет, то ему следует включить в перечень хотя бы один криптонабор, алгоритм хэширования которого ассоциирован с секретом. Сервер должен игнорировать криптонаборы, которые он не распознает, не поддерживает или не желает использовать, и обрабатывать оставшиеся криптонаборы без учета игнорирования;

- `legacy_compression_methods` — устаревшее поле со списком алгоритмов сжатия, которые поддерживает клиент. Список должен быть задан одним байтом, и этот байт должен быть нулевым. При нарушении данного условия сервер должен закрыть соединение с оповещением `illegal_parameter`;

- `extensions` — перечень расширений. Тип `Extension` определен в 6.3.1. Имеются обязательные расширения. В частности, в перечень должно входить расширение `supported_versions`. Сервер должен игнорировать нераспознанные расширения.

Клиент должен отправить `ClientHello` при подключении к серверу. Клиент повторно отправляет `ClientHello`, если в ответ на первое сообщение он получил от сервера сообщение `HelloRetryRequest`. Повторное сообщение `ClientHello` должно быть получено из первого следующими корректировками.

- 1 Исправление расширения `key_share`, если оно включено в `HelloRetryRequest`. Исправленное расширение должно включать единственную запись `KeyShareEntry`, и в этой записи должен быть указан открытый ключ для выбранной сервером циклической группы.

- 2 Исключение расширения `early_data` (см. 6.3.12), если оно включено. При перезапуске Handshake через `HelloRetryRequest` защита ранних прикладных данных запрещена.

- 3 Включение расширения `cookie` (см. 6.3.3), если оно включено в `HelloRetryRequest`.

- 4 Обновление расширения `pre_shared_key` (см. 6.3.13), если оно включено. В расширении пересчитываются поля `obfuscated_ticket_age` и PSK-скрепки. Могут исключаться идентификаторы PSK, алгоритмы хэширования которых не соответствуют криптонабору, выбранному сервером.

- 5 Включение или исключение расширения `padding` (см. таблицу 1), а также изменение его размера.

- 6 Корректировка в соответствии с расширением, которое определяется за пределами настоящего стандарта и которое включается в `HelloRetryRequest`.

Другие корректировки `ClientHello` не допускаются.

Если сервер получает `ClientHello` в неоговоренные моменты времени, то он должен прервать Handshake с оповещением `unexpected_message`.

Если сервер установил соединение TLS версии 1.2 и в рамках переустановки связи получает `ClientHello` версии 1.3, то сервер должен сохранить версию 1.2 соединения и не переходить на версию 1.3.

Если клиент запрашивает с помощью какого-либо расширения дополнительные функциональные возможности, а сервер эти возможности не предоставляет, то клиент может прервать Handshake.

После отправки `ClientHello` клиент ожидает сообщения `ServerHello` или `HelloRetryRequest`. Если используется механизм 0-RTT, то в процессе ожидания клиент может передавать ранние прикладные данные.

6.2.3 Сообщение `ServerHello`

Сервер отправляет сообщение `ServerHello`, если присланное `ClientHello` устраивает сервер, позволяет ему согласовать параметры и сформировать общие ключи.

Формат сообщения `ServerHello` описывается одноименным типом:

```
struct {
    ProtocolVersion legacy_version = 0x0303;    /* TLS v1.2 */
    Random random;
    opaque legacy_session_id_echo<0..32>;
    CipherSuite cipher_suite;
    uint8 legacy_compression_method = 0;
    Extension extensions<6..216-1>;
} ServerHello;
```

Поля `ServerHello` имеют следующее значение:

- `legacy_version` — устаревшее поле для номера версии TLS. В поле должно быть установлено значение `0x0303`. Клиент должен проверить данное условие и закрыть соединение с оповещением `illegal_parameter` при его нарушении;
- `random` — случайные данные сервера. Должны генерироваться независимо от случайных данных клиента в `ClientHello.random`. При выборе TLS версии 1.2 последние 8 байт `random` должны быть заданы специальным образом, как это объясняется ниже;
- `legacy_session_id_echo` — повтор поля `legacy_session_id` из сообщения `ClientHello` клиента. Клиент должен прервать Handshake с оповещением `illegal_parameter`, если `legacy_session_id_echo` отличается от `legacy_session_id`;
- `cipher_suite` — криптонабор, выбранный сервером из перечня `ClientHello.cipher_suites`. Клиент должен прервать Handshake с оповещением `illegal_parameter`, если выбранный сервером криптонабор не был указан в перечне клиента;
- `legacy_compression_method` — список алгоритмов сжатия. Должен быть задан одним байтом и этот байт должен быть нулевым. При нарушении данного условия клиент должен прервать Handshake с оповещением `illegal_parameter`;
- `extensions` — перечень расширений. Должен содержать только те расширения, которые нужны для согласования параметров и формирования общих ключей. Другие расширения могут передаваться в сообщении `EncryptedExtensions`. Перечень должен включать расширение `supported_groups`, а также одно или оба расширения `pre_shared_key` и `key_share` (см. 6.2.1). В перечень не должны входить расширения, которые не были включены в `ClientHello`.

Для обратной совместимости с промежуточными сетевыми устройствами (middleboxes) сообщение `HelloRetryRequest` имеет такой же формат, как `ServerHello`. Единственное отличие состоит в том, что в поле `random` сообщения `HelloRetryRequest` всегда устанавливается значение

CF21AD74E59A6111BE1D8C021E65B891C2A211167ABB8C5E079E09E2C8A8339C₁₆.

Примечание — Получено применением алгоритма хэширования SHA-256 к строке "HelloRetryRequest".

При приеме сообщения Handshake с типом `server_hello` клиент должен отличить `ServerHello` от `HelloRetryRequest` на основании поля `random`. Если в поле установлено указанное значение, то сообщение обрабатывается как `HelloRetryRequest`.

Если сервер согласовал использование TLS версии 1.2, то для защиты от атак по понижению версии он должен указать в последних 8 байтах `ServerHello.random` следующее значение:

444F574E47524401₁₆.

При согласовании TLS версии 1.2 клиент должен проверить, что последние 8 байт `random` отличаются от указанных выше. Если условие не выполнено, т. е. сервер следует настоящему стандарту и, таким образом, поддерживает TLS версии 1.3, то клиент должен прервать Handshake с оповещением `illegal_parameter`.

Клиент TLS, который переустанавливает связь в соответствии с TLS версии 1.2 и при этом получает `ServerHello` версии 1.3, должен прервать Handshake с оповещением `protocol_version`.

6.2.4 Сообщение HelloRetryRequest

Сервер отправляет сообщение `HelloRetryRequest`, если присланное `ClientHello` позволяет согласовать параметры и сформировать общие ключи, но не содержит всех необходимых для этого сведений.

Сообщение `HelloRetryRequest` имеет такой же формат, как `ServerHello`, повторяется семантика полей `legacy_version`, `legacy_session_id_echo`, `cipher_suite` и `legacy_compression_method`. Тем не менее, в настоящем стандарте `HelloRetryRequest` считается отдельным сообщением.

Сообщение `HelloRetryRequest` должно содержать расширение `supported_versions` (см. 6.3.2). Серверу следует дополнительно включить в `HelloRetryRequest` минимальный набор расширений, который позволит скорректировать `ClientHello`. Как и для `ServerHello`, сообщение `HelloRetryRequest` не должно содержать расширений, которые не были включены в `ClientHello`. Единственным исключением является необязательное расширение `cookie` (см. 6.3.3).

Кроме `supported_versions` и `cookie` в настоящем стандарте разрешается включать в `HelloRetryRequest` только расширение `key_share` (см. 6.3.10).

Получив `HelloRetryRequest`, клиент должен проверить поля `legacy_version`, `legacy_session_id_echo`, `cipher_suite` и `legacy_compression_method` по правилам, установленным в 6.2.3. После этого клиент должен обработать расширения, начиная с `supported_versions`. Клиент должен прервать Handshake с оповещением `illegal_parameter`, если учет расширений `HelloRetryRequest` не изменит `ClientHello`.

Если клиент получает второе сообщение `HelloRetryRequest` в том же соединении, т. е. после своего повторного `ClientHello`, то он должен прервать рукопожатие с оповещением `unexpected_message`.

При успешной обработке `HelloRetryRequest` клиент должен отправить повторное сообщение `ClientHello`.

Клиент должен прервать Handshake, если выбранный сервером криптонабор не был указан в перечне клиента. Сервер должен повторить свой выбор в `ServerHello` после повторного `ClientHello` (если сервер выбирает криптонабор в первую очередь, то это будет происходить автоматически). Клиент должен проверить, что в `ServerHello` указан

тот же криптонабор, что в `HelloRetryRequest`. Клиент должен прервать Handshake с оповещением `illegal_parameter` при нарушении данного условия.

Клиенту следует исключить из повторного `ClientHello` те предварительно согласованные секреты, которые не соответствуют алгоритму хэширования выбранного криптонабора.

Клиент должен проверить, что версия TLS, указанная сервером в сообщении `HelloRetryRequest` через расширение `supported_versions`, повторяется в `ServerHello`. Клиент должен прервать Handshake с оповещением `illegal_parameter` при нарушении данного условия.

6.3 Расширения

6.3.1 Назначение и перечень

С помощью расширений клиент и сервер согласовывают параметры, обмениваются ключевым материалом. Расширение представляет собой структуру типа `Extension`, который определяется следующим образом:

```
struct {
    ExtensionType extension_type;
    opaque extension_data<0..216-1>;
} Extension;
```

Поля `Extension` имеют следующее значение:

- `extension_type` — тип расширения;
- `extension_data` — данные расширения, определяемые в зависимости от типа.

Вложенный тип `ExtensionType` определяется следующим образом:

```
enum {
    server_name(0),
    max_fragment_length(1),
    status_request(5),
    supported_groups(10),
    signature_algorithms(13),
    use_srtp(14),
    heartbeat(15),
    application_layer_protocol_negotiation(16),
    signed_certificate_timestamp(18),
    client_certificate_type(19),
    server_certificate_type(20),
    padding(21),
    pre_shared_key(41),
    early_data(42),
    supported_versions(43),
    cookie(44),
    psk_key_exchange_modes(45),
    certificate_authorities(47),
    oid_filters(48),
    post_handshake_auth(49),
    signature_algorithms_cert(50),
    key_share(51),
```

```
(65535)
} ExtensionType;
```

Большинство расширений имеют две формы: запрос и ответ. Имеются расширения информационного характера, которые не предполагают ответа. Запрос клиента отправляется в сообщении `ClientHello`, соответствующий ответ сервера — в `ServerHello`, `EncryptedExtensions`, `HelloRetryRequest` или `Certificate`. В обратную сторону, запрос сервера отправляется в `CertificateRequest`, клиент может дать на него ответ в `Certificate`. Сервер может отправить расширение, не предполагающее ответа, в `NewSessionTicket`.

Сторона TLS не должна отправлять расширение-ответ, если не получено расширение-запрос. При получении ответа без запроса протокол Handshake должен быть прерван с оповещением `unexpected_message`. Исключением из данного правила является отправка расширения `cookie` в сообщении `HelloRetryRequest` (см. 6.3.3). В таблице 1 перечислены расширения TLS версии 1.3. Часть расширений вводится в настоящем стандарте, остальные — в спецификациях, ссылки на которые даны в квадратных скобках. В таблице 1 указаны сообщения Handshake, в которых расширения пересылаются. Используются сокращения: CH — `ClientHello`, SH — `ServerHello`, EE — `EncryptedExtensions`, CT — `Certificate`, CR — `CertificateRequest`, NST — `NewSessionTicket` и HRR — `HelloRetryRequest`. Расширения перечисляются в порядке возрастания их типов в `ExtensionType`. Расширения, помеченные звездочкой, должны быть реализованы обязательно, если противное не оговаривается за пределами настоящего стандарта относительно конкретного случая применения TLS.

Следующие расширения не вводятся в настоящем стандарте, но учитываются при определении механизмов TLS.

- 1 В расширении `server_name` клиент передает имя сервера, с которым он собирается взаимодействовать. Имя учитывается при выборе сертификата для аутентификации. Расширение оказывается полезным в ситуации, когда один TLS-сервер обслуживает несколько подчиненных прикладных серверов (возможно виртуальных).

- 2 Расширение `max_fragment_length` используется для согласования максимальной длины фрагментов данных при их передаче по протоколу Record.

- 3 Расширение `application_layer_protocol_negotiation` используется для согласования прикладного протокола, который будет выполняться поверх TLS.

- 4 Расширения `client_certificate_type` и `server_certificate_type` используются для согласования типа сертификата для аутентификации клиента и сервера соответственно. Поддерживаются два типа сертификатов: обычные сертификаты X.509, определенные в СТБ 34.101.19, и контейнеры типа `SubjectPublicKeyInfo` ACH.1, также определенного в СТБ 34.101.19. Контейнер содержит открытый ключ, но не содержит ни идентификационных данных, ни срока действия, ни подписи УЦ. Обмен открытыми ключами в облегченных контейнерах востребован в некоторых сценариях использования TLS (см. [6]).

- 5 Расширение `padding` используется для искусственного увеличения размера сообщения `ClientHello` через включение в расширение незначащих нулевых байтов. Известны реализации TLS, которые некорректно обрабатывают `ClientHello` определенных длин. Увеличение размера сообщений помогает избежать ошибок при выполнении TLS с такими реализациями.

Если сторона получает расширение в непредназначенном для него сообщении, то она должна прервать Handshake с оповещением `illegal_parameter`.

Таблица 1 — Расширения

Расширение	Определение	Сообщения Handshake
server_name*	[7]	CH, EE
max_fragment_length	[7]	CH, EE
status_request	6.3.7	CH, CR, CT
supported_groups*	6.3.9	CH, EE
signature_algorithms*	6.3.4	CH, CR
use_srtp	[8]	CH, EE
heartbeat	[9]	CH, EE
application_layer_protocol_negotiation	[10]	CH, EE
signed_certificate_timestamp	[11]	CH, CR, CT
client_certificate_type	[6]	CH, EE
server_certificate_type	[6]	CH, EE
padding	[12]	CH
pre_shared_key	6.3.13	CH, SH
early_data	6.3.12	CH, EE, NST
supported_versions*	6.3.2	CH, SH, HRR
cookie*	6.3.3	CH, HRR
psk_key_exchange_modes	6.3.11	CH
certificate_authorities	6.3.5	CH, CR
oid_filters	6.3.6	CR
post_handshake_auth	6.3.8	CH
signature_algorithms_cert*	6.3.4	CH, CR
key_share*	6.3.10	CH, SH, HRR

Если сообщение содержит несколько расширений, то они могут идти в любом порядке. Исключение составляет расширение **pre_shared_key**, которое должно быть последним в **ClientHello** (хотя может находиться в произвольном месте списка расширений **ServerHello**).

Согласование параметров с помощью расширений выполняется в каждом сеансе Handshake, в том числе при возобновлении связи. Однако параметры секрета, который будет использоваться в механизме 0-RTT, изменить нельзя: они были согласованы в предыдущем соединении. При несоответствии параметров секрета параметрам текущего соединения механизм 0-RTT может быть отменен (см. 6.3.12).

Перечень расширений может расширяться. При разработке новых расширений следует учитывать следующее:

- сервер может не принять расширение, предложенное клиентом, как из-за ошибки, так и просто отказав в поддержке определенных функциональных возможностей. В первом случае следует прерывать Handshake с критическим оповещением, во втором — объяснять отказ в ответной части расширения;
- вплоть до завершения Handshake противник может манипулировать сообщениями протокола, вставляя, удаляя или перемещая расширения. Расширения следует проектировать так, чтобы противник не мог навязать использование (или, наоборот, отключение) определенных функциональных возможностей TLS.

6.3.2 Расширение `supported_versions`

Клиент использует расширение `supported_versions` для информирования о поддерживаемых версиях TLS, сервер — для информирования о выбранной версии.

Данные расширения `supported_versions` описываются типом `SupportedVersions`, который определяется следующим образом:

```
struct {
    select (Handshake.msg_type) {
        case client_hello:
            ProtocolVersion versions<2..254>;
        case server_hello: /* and HelloRetryRequest */
            ProtocolVersion selected_version;
    };
} SupportedVersions;
```

В поле `versions` клиент перечисляет поддерживаемые версии в порядке убывания предпочтения. Сервер указывает выбранную версию в `selected_version`.

Клиент должен включить расширение `supported_versions` в сообщение `ClientHello`, указав в расширении список версий TLS, которые он готов использовать. Версия 1.3, представленная числом `0x0304`, должна входить в список.

Если сервер получил `ClientHello` без расширения `supported_versions` и если сервер поддерживает TLS версии 1.2, то для согласования версии он должен использовать схему, заданную в СТБ 34.101.65. Данное правило действует и тогда, когда поле `ClientHello.legacy_version` содержит значение `0x0304` или больше, т. е. клиент предлагает использовать TLS версии 1.3 или выше и при этом не отправляет `supported_versions`. Сервер может прервать `Handshake`, если поле `ClientHello.legacy_version` содержит значение `0x0304` или больше.

Если сервер получил `ClientHello` с расширением `supported_versions`, то он должен игнорировать `ClientHello.legacy_version` и при выборе версии использовать исключительно расширение. Сервер должен выбирать из версий, перечисленных в расширении, и при этом игнорировать версии, которые ему неизвестны, а также версии ниже 1.2. Допускается, что будет выбрана версия 1.2. Сервер должен быть готов к обработке сообщения `ClientHello`, в расширении `supported_versions` которого отсутствует версия `0x0304`.

Если сервер выбрал версию 1.2, то он должен указать эту версию в поле `ServerHello.legacy_version` (`ServerHello.version` в TLS версии 1.2) и не должен включать в `ServerHello` расширение `supported_versions`. Если сервер выбрал версию 1.3, то расширение должно быть включено и в нем должна быть указана выбранная версия (`0x0304`). Кроме этого, сервер должен установить в `ServerHello.legacy_version` значение `0x0303` (TLS версии 1.2).

Клиент должен проверять наличие `supported_versions` в `ServerHello`. Если расширение присутствует, то клиент должен игнорировать `ServerHello.legacy_version` и при определении версии использовать исключительно расширение. Если в расширении указана версия ниже 1.3 или версия, которую клиент не предлагал, то клиент должен прервать `Handshake` с оповещением `illegal_parameter`.

При перезапуске `Handshake` сервер должен проинформировать о выборе версии в сообщении `HelloRetryRequest` точно так же, как он это делает в `ServerHello`.

6.3.3 Расширение cookie

Куки (cookie) — это данные, которые сервер отправляет клиенту на хранение. В куки сервер передает клиенту состояние, избегая необходимости хранить его самостоятельно. Кроме этого, сервер может проверить достижимость сетевого адреса клиента (защита от атак «отказ в обслуживании»), предлагая тому возвратить присланные куки.

Данные расширения cookie описываются типом `Cookie`, который определяется следующим образом:

```
struct {
    opaque cookie<1..216-1>;
} Cookie;
```

В поле `cookie` содержатся куки.

Сервер может отправить расширение cookie в сообщении `HelloRetryRequest`. После получения `HelloRetryRequest` с расширением cookie клиент должен перенести расширение в повторное сообщение `ClientHello`. Клиент не должен включать расширение в первое `ClientHello` в последующих соединениях с сервером.

Примечание — Отправка cookie в `HelloRetryRequest` является исключением из общего правила, согласно которому сервер может отправлять клиенту только те расширения, которые были включены в `ClientHello` (см. 6.3.1).

Сервер может указать в cookie защищенное хэш-значение первого сообщения `ClientHello`. Получив повторное `ClientHello`, сервер снимает защиту, определяет хэш-значение и использует его при обработке стенограммы `Handshake` (см. 9.2). При такой схеме взаимодействия серверу не нужно хранить первое `ClientHello`.

Примечание — При определенных условиях сервер может реконструировать первое `ClientHello` по повторному, а затем проверить, что хэш-значение реконструированного сообщения совпадает с присланным в cookie.

Если сервер работает без состояния, то он не может отличить первое сообщение `ClientHello` от повторного. Между этими сообщениями сервер может получить незащищенную запись типа `change_cipher_spec`. Сервер без состояния должен игнорировать эту запись (см. 7.1).

6.3.4 Расширения `signature_algorithms` и `signature_algorithms_cert`

Клиент и сервер используют расширения `signature_algorithms` и `signature_algorithms_cert` для информирования о поддерживаемых алгоритмах ЭЦП. Первое расширение касается подписи в сообщении `CertificateVerify`, второе — подписей сертификатов.

Данные расширений `signature_algorithms` и `signature_algorithms_cert` описываются типом `SignatureSchemeList`, который определяется следующим образом:

```
struct {
    SignatureScheme supported_signature_algorithms<2..216-2>;
} SignatureSchemeList;
```

В поле `supported_signature_algorithms` перечисляются идентификаторы алгоритмов ЭЦП в порядке убывания предпочтения.

Идентификаторы описываются типом `SignatureScheme`, который определяется следующим образом:

```
enum { (0xFFFF) } SignatureScheme;
```

В [3] в `SignatureScheme` включены идентификаторы некоторых алгоритмов ЭЦП. Дополнительные идентификаторы включаются в В.5.

Если аутентификация сервера проводится по его сертификату, т. е. основана на проверке владения соответствующим личным ключом, то клиент должен включить `signature_algorithms` в сообщение `ClientHello`. Аналогичным образом, если сервер запрашивает аутентификацию клиента по сертификату, высылая тому сообщение `CertificateRequest`, то это сообщение должно содержать расширение `signature_algorithms`.

Если сторона аутентифицируется по сертификату, но не получила `signature_algorithms` в `ClientHello` или `CertificateRequest`, то она должна прервать `Handshake` с оповещением `missing_extension`.

Открытый ключ сертификата, который используется для аутентификации, должен соответствовать одному из алгоритмов, указанных в `signature_algorithms`.

Расширение `signature_algorithms` может сопровождаться расширением `signature_algorithms_cert`. В последнем расширении перечисляются алгоритмы, с помощью которых проверяются подписи в цепочке сертификатов аутентифицируемой стороны. Если расширение `signature_algorithms_cert` опущено, то перечень алгоритмов определяется по `signature_algorithms`.

Алгоритмы проверки подписей в цепочке сертификатов должны входить в перечень `signature_algorithms_cert` или, если это расширение опущено, в перечень `signature_algorithms`. Требование не распространяется на начальные сертификаты цепочек (точки доверия, как правило, корневые сертификаты), подписи которых могут не проверяться.

6.3.5 Расширение `certificate_authorities`

Расширение `certificate_authorities` используется для информирования об УЦ, которым отправитель доверяет. Получателю следует учитывать информацию при выборе сертификата для аутентификации.

Данные расширения `certificate_authorities` описываются типом `CertificateAuthorities`, который определяется следующим образом:

```
opaque DistinguishedName<1..216-1>;

struct {
    DistinguishedName authorities<3..216-1>;
} CertificateAuthoritiesExtension;
```

В поле `authorities` указывается список имен доверенных УЦ. Имя описывается типом `Name ASN.1`, который определен в СТБ 34.101.19, и кодируется по правилам DER. В качестве доверенного УЦ может указываться как корневой центр (точка доверия), так и промежуточный.

Клиент может отправить расширение `certificate_authorities` в сообщении `ClientHello`, сервер — в сообщении `CertificateRequest`.

6.3.6 Расширение `oid_filters`

В расширении `oid_filters` сервер указывает требования к сертификатам, которые используются для аутентификации перед ним. Требования касаются расширений сертификатов, определенных в СТБ 34.101.19, и формулируются в виде пар «идентификатор расширения — требуемое содержание».

Данные расширения `oid_filters` описываются типом `OIDFilterExtension`, который определяется следующим образом:

```
struct {
    opaque certificate_extension_oid<1..28-1>;
    opaque certificate_extension_values<0..216-1>;
} OIDFilter;

struct {
    OIDFilter filters<0..216-1>;
} OIDFilterExtension;
```

В поле `filters` указывается список элементов типа `OIDFilter`. Каждый элемент является отдельным расширением сертификата и состоит из двух полей:

- `certificate_extension_oid` — идентификатор расширения. Описывается типом «идентификатор объекта» (OBJECT IDENTIFIER) АСН.1;
- `certificate_extension_values` — список требуемых значений расширения. Определяется в соответствии со спецификацией расширения в СТБ 34.101.19.

Оба поля кодируются по правилам DER.

Если сервер использует расширение `oid_filters`, то он должен включить его в сообщение `CertificateRequest`. Идентификаторы расширений сертификата в `oid_filters` не должны повторяться.

Если сервер включил `oid_filters` в `CertificateRequest`, то ответный сертификат клиента должен содержать все расширения, перечисленные в `oid_filters` и распознаваемые клиентом. В этих расширениях должны присутствовать все требуемые значения. Клиент должен игнорировать перечисленные в `oid_filters` расширения сертификатов, идентификаторы которых он не распознал. Если клиент проигнорировал некоторые расширения сертификатов и представил сертификат, который не полностью соответствует требованиям `oid_filters`, то сервер может либо продолжить Handshake без аутентификации клиента, либо прервать протокол с оповещением `unsupported_certificate`.

Способ проверки требований к расширению сертификата меняется в зависимости от типа расширения. В настоящем стандарте определяются правила проверки следующих расширений.

1 Расширение `KeyUsage` в сертификате клиента соответствует требованиям `oid_filters`, если каждый бит назначения ключа, установленный в `oid_filters`, установлен также в сертификате.

2 Расширение `ExtKeyUsage` (название в соответствии с СТБ 34.101.78) в сертификате клиента соответствует требованиям `oid_filters`, если каждый идентификатор области применения, указанный в `oid_filters`, указан также в сертификате. В `oid_filters` не должен указываться идентификатор `anyExtendedKeyUsage`.

За пределами настоящего стандарта могут устанавливаться правила проверки требований к другим расширениям сертификата.

6.3.7 Расширение `status_request`

Расширение `status_request` используется для проверки статуса сертификата, используемого для аутентификации. Статус проверяется с помощью протокола OCSP, установленного в СТБ 34.101.26.

Данные расширения `status_request` описываются типом `StatusRequestExtension`, который определяется следующим образом:

```

enum { ocsp(1), (255) } CertificateStatusType;

opaque ResponderID<1..216-1>;
opaque Extensions<0..216-1>;

struct {
    ResponderID responder_id_list<0..216-1>;
    Extensions request_extensions;
} OCSPStatusRequest;

struct {
    CertificateStatusType status_type = ocsp;
    OCSPStatusRequest request;
} CertificateStatusRequest;

opaque OCSPResponse<1..224-1>;

struct {
    CertificateStatusType status_type = ocsp;
    OCSPResponse response;
} CertificateStatus;

struct {} Empty;

struct {
    select (Handshake.msg_type) {
        case client_hello:
            CertificateStatusRequest request;
        case certificate_request: Empty;
        case certificate:
            CertificateStatus status;
    };
} StatusRequestExtension;

```

Вложенный тип `OCSPStatusRequest` описывает OCSP-запросы, тип `OCSPResponse` — OCSP-ответ. Поля `OCSPStatusRequest` имеют следующее значение:

- `responder_id_list` — список имен доверенных OCSP-серверов. Пустой список означает, что доверенные OCSP-серверы неявно известны, например, предварительно согласованы;
- `request_extensions` — OCSP-запросы.

OCSP-серверы, запросы и ответы описываются типами `ResponderID`, `OCSPRequest` и `OCSPResponse` АСН.1, определенными в СТБ 34.101.26. Данные АСН.1 кодируются по правилам DER.

Если клиент использует расширение `status_request`, то он должен включить его в сообщение `ClientHello`.

Получив расширение `status_request`, сервер в сообщении `Certificate` вместе со своим сертификатом может выслать OCSP-ответ со сведениями о его статусе. Для этого сервер формирует расширение `status_request` с OCSP-ответом и включает его в перечень

расширений своего сертификата. При подготовке OSCP-ответа серверу следует использовать сведения в присланном расширении `status_request`: перечни OSCP-серверов и OSCP-запросов.

Сервер может запросить статус сертификата клиента, включив пустое расширение `status_request` в сообщение `CertificateRequest`. Получив запрос, клиент в сообщении `Certificate` вместе со своим сертификатом может выслать расширение `status_request` с OSCP-ответом о статусе сертификата.

6.3.8 Расширение `post_handshake_auth`

Расширение `post_handshake_auth` используется для информирования о том, что клиент готов выполнить аутентификацию после Handshake (см. 6.7.2).

Расширение `post_handshake_auth` не содержит данных. Формально данные описываются пустым типом `PostHandshakeAuth`:

```
struct {} PostHandshakeAuth;
```

Если клиент использует расширение `post_handshake_auth`, то он должен включить его в сообщение `ClientHello`. Если сервер получил `ClientHello` без расширения, то он не должен отправлять клиенту сообщение `CertificateRequest` по завершении Handshake.

Сервер не должен отправлять расширение `post_handshake_auth`.

6.3.9 Расширение `supported_groups`

В расширении `supported_groups` отправитель перечисляет циклические группы, которые он может использовать в протоколе Диффи – Хеллмана.

Данные расширения `supported_groups` описываются типом `NamedGroupList`, который определяется следующим образом:

```
struct {
    NamedGroup named_group_list <2..216-2>;
} NamedGroupList;
```

В поле `named_group_list` указываются имена поддерживаемых групп в порядке убывания предпочтения.

Имена групп описываются типом `NamedGroup`, который определяется следующим образом:

```
enum { (0xFFFF) } NamedGroup;
```

В [3] в `NamedGroup` включены имена некоторых групп. Дополнительные имена включаются в В.4.

Клиент включает расширение `supported_groups` в сообщение `ClientHello`, если он планирует использовать протокол Диффи – Хеллмана (режим DHE, см. 6.2.1) для формирования общих ключей.

Сервер может включить расширение `supported_groups` в сообщение `EncryptedExtensions`. Клиент не должен использовать сведения в расширении сервера вплоть до завершения сеанса Handshake, но может использовать их в следующих сеансах для выбора групп в своих расширениях `key_share`. Если у сервера есть более предпочтительные группы, нежели те, что указаны в `key_share` клиента, то серверу следует выслать расширение `supported_groups`, проинформировав тем самым клиента о своих предпочтениях. При этом серверу следует указать в `supported_groups` все поддерживаемые группы, независимо от того, поддерживаются они клиентом или нет.

6.3.10 Расширение `key_share`

Расширение `key_share` содержит данные обмена между сторонами в процессе выполнения протокола Диффи – Хеллмана.

Единица данных описывается типом `KeyShareEntry`, который определяется следующим образом:

```
struct {
    NamedGroup group;
    opaque key_exchange <1..216-1>;
} KeyShareEntry;
```

Поля `KeyShareEntry` имеют следующее значение:

- `group` — имя циклической группы, которая используется в протоколе Диффи – Хеллмана;
- `key_exchange` — одноразовый открытый ключ протокола. Формат открытого ключа определяется в зависимости от используемой циклической группы. Форматы открытых ключей для некоторых известных групп определены в [3], а также в В.3.

Клиент отправляет расширение `key_share` в сообщении `ClientHello` вместе с расширением `supported_groups` (см. 6.2.2).

Данные расширения `key_share` в `ClientHello` описываются типом `KeyShareClientHello`, который определяется следующим образом:

```
struct {
    KeyShareEntry client_shares <0..216-1>;
} KeyShareClientHello;
```

В поле `client_shares` клиент указывает перечень элементов типа `KeyShareEntry` в порядке убывания предпочтения. Элементы должны соответствовать группам, перечисленным в `supported_groups`, и идти в той же очередности. При этом некоторые группы, в том числе самые предпочтительные, могут быть не представлены элементами `client_shares`. Такая ситуация возникает, например, тогда, когда предпочтительная группа является новой, скорее всего не будет поддержана сервером и поэтому клиенту нецелесообразно заранее вычислять открытый ключ для этой группы.

Примечание — Тот факт, что клиент включил в `KeyShareClientHello` открытый ключ для группы *A* и не включил для группы *B*, не означает, что клиент предпочитает группу *A* группе *B*. Поэтому выбор группы на основании `KeyShareClientHello` может дать в качестве результата не самую предпочтительную группу из тех, которые клиент и сервер одновременно поддерживают. Если серверу требуется максимально точно учесть предпочтения клиента, то ему следует сначала выбрать группу на основании `supported_groups`, а затем, в зависимости от того, поддержана ли группа открытым ключом в `KeyShareClientHello`, либо предложить использовать группу через `HelloRetryRequest`, либо выслать свой открытый ключ для группы в `ServerHello`.

Клиент может включить в `KeyShareClientHello` открытый ключ для каждой группы, указанной в `supported_groups`. Открытые ключи должны генерироваться независимо друг от друга. Клиент не должен включать в `KeyShareClientHello` несколько открытых ключей для одной и той же группы. Клиент не должен включать в `KeyShareClientHello` открытые ключи для групп, не указанных в `supported_groups`. Сервер может проверять эти требования и прерывать `Handshake` с оповещением `illegal_parameter` при нарушении любого из них.

Клиент может отправить пустой список `KeyShareClientHello.client_shares`, запрашивая тем самым перезапуск `Handshake` с выбором группы со стороны сервера. Сервер

инициирует перезапуск самостоятельно, если ни одна из групп, указанных в списке, его не устраивает. Перезапуск реализуется отправкой сообщения `HelloRetryRequest` с включением в него расширения `key_share`.

Данные расширения `key_share` в `HelloRetryRequest` описываются типом `KeyShareHelloRetryRequest`, который определяется следующим образом:

```
struct {
    NamedGroup selected_group;
} KeyShareHelloRetryRequest;
```

В поле `selected_group` сервер указывает имя подходящей группы, выбирая среди групп, указанных клиентом в расширении `supported_groups`.

Получив расширение `key_share` в `HelloRetryRequest`, клиент должен проверить два условия:

- 1) в `selected_group` указана одна из групп списка `supported_groups`;
- 2) в `selected_group` не указана группа, для которой в расширении `key_share` предыдущего сообщения `ClientHello` был выслан открытый ключ.

При нарушении любого из условий клиент должен прервать Handshake с оповещением `illegal_parameter`. Если условия соблюдены, то клиент отправляет повторное сообщение `ClientHello`. В нем клиент должен изменить расширение `key_share`. В новую редакцию расширения должен быть включен только один элемент типа `KeyShareEntry`, и он должен содержать открытый ключ для группы, указанной в `KeyShareHelloRetryRequest.selected_group`.

Если расширение `key_share` клиента устраивает сервер и он может продолжить Handshake без перезапуска, то сервер отвечает сообщением `ServeHello`, также включая в него `key_share`.

Данные расширения `key_share` в `ServerHello` описываются типом `KeyShareServerHello`, который определяется следующим образом:

```
struct {
    KeyShareEntry server_share;
} KeyShareServerHello;
```

В поле `server_share` сервер указывает свой одноразовый открытый ключ.

Открытый ключ сервера должен принадлежать одной из групп, для которых клиент представил открытый ключ в своем расширении `key_share`. Сервер не должен отправлять открытый ключ, если клиент включил в `ClientHello` расширение `psk_key_exchange_modes`, и в этом расширении установлен режим `psk_ke` (см. 6.3.11). Если выполнялся перезапуск Handshake и до `ServerHello` сервер отправлял сообщение `HelloRetryRequest`, то клиент должен проверить, что открытый ключ сервера в `KeyShareServerHello` принадлежит группе, указанной в `KeyShareHelloRetryRequest`. При нарушении этого условия клиент должен прервать Handshake с оповещением `illegal_parameter`.

6.3.11 Расширение `psk_key_exchange_modes`

В расширении `psk_key_exchange_modes` клиент сообщает, какие режимы формирования общих ключей с участием PSK он поддерживает. Расширение касается как внешнего PSK, согласованного за пределами TLS, так и PSK для возобновления связи, полученного ранее от сервера в сообщении `NewSessionTicket`.

Данные расширения `psk_key_exchange_modes` описываются типом `PskKeyExchangeModes`, который определяется следующим образом:

```
enum { psk_ke(0), psk_dhe_ke(1), (255) } PskKeyExchangeMode;

struct {
    PskKeyExchangeMode ke_modes<1..255>;
} PskKeyExchangeModes;
```

В поле `ke_modes` указываются допустимые режимы:

- `psk_ke` — режим PSK. В этом режиме сервер не должен отправлять расширение `key_share`;
- `psk_dhe_ke` — режим PSK+DHE. В этом режиме клиент и сервер должны обмениваться расширениями `key_share`.

За пределами настоящего стандарта могут определяться дополнительные режимы. Это должно быть сделано так, чтобы по сообщениям Handshake можно было однозначно определить, какой режим выбран.

Примечание — Выбор `psk_dhe_ke` подтверждается включением расширения `key_share` в `ServerHello`, выбор `psk_ke` — невключением.

Если клиент включает в `ClientHello` расширение `pre_shared_key`, то он должен сопровождать его расширением `psk_key_exchange_modes`. Если сервер получает `ClientHello` с `pre_shared_key`, но без `psk_key_exchange_modes`, то он должен прервать Handshake. Сервер не должен использовать режим формирования общих ключей, отличный от заданных клиентом. Серверу не следует отправлять в `NewSessionTicket` секрет PSK, предназначенный для использования в режимах, которые не поддерживает клиент. Попытка возобновления связи с таким секретом приведет к ошибке.

Сервер не должен отправлять расширение `psk_key_exchange_modes`.

6.3.12 Расширение `early_data`

Расширение `early_data` используется для информирования о применении механизма 0-RTT (см. 5.6) и для передачи его настроек.

Данные расширения `extension_data` описываются типом `EarlyDataIndication`, который определяется следующим образом:

```
struct {} Empty;

struct {
    select (Handshake.msg_type) {
        case new_session_ticket:
            uint32 max_early_data_size;
        case client_hello: Empty;
        case encrypted_extensions: Empty;
    };
} EarlyDataIndication;
```

Если имеется секрет PSK, который годится для использования в механизме 0-RTT, то клиент может применить механизм. Для этого клиент должен включить в `ClientHello` расширения `pre_shared_key` и `early_data`. В первом расширении клиент описывает секрет, во втором информирует об его использовании для защиты ранних прикладных данных. В `pre_shared_key` используемый секрет должен быть описан первым.

Параметры 0-RTT определяются атрибутами используемого секрета. При согласовании секрета за пределами TLS атрибуты определяются в момент согласования. Если сек-

рет был согласован в предыдущем соединении, то атрибуты определяются параметрами и сообщениями этого соединения.

При согласовании секрета в предыдущем соединении сервер высылает клиенту сообщение **NewSessionTicket**. Это сообщение должно содержать расширение **early_data**. В поле **EarlyDataIndication.max_early_data_size** расширения должен быть указан лимит на объем ранних прикладных данных (см. 6.7.1).

При использовании механизма 0-RTT сообщения Handshake и прикладные данные первой (ранней) очереди имеют тот же формат и защищаются так же, как сообщения и данные последующих очередей. Единственное отличие состоит в том, что при защите ранних прикладных данных используются специальные ключ и начальная синхропосылка, построенные по **client_early_traffic_secret** (см. 9.4). Если сервер согласился с использованием механизма 0-RTT, то после приема его сообщения **Finished** клиент отправляет в ответ сообщение **EndOfEarlyData**, информируя тем самым о завершении ранней защиты и смене ключей. Сообщение **EndOfEarlyData** защищается так же, как ранние прикладные данные.

Сервер, который получает сообщение **ClientHello** с расширением **early_data**, должен поступить одним из следующих способов.

1 Сервер игнорирует расширение **early_data** и действует по правилам 1-RTT, пропуская ранние прикладные данные клиента. Для этого сервер обрабатывает присылаемые защищенные записи **Record**, которым в сумме соответствует не более **max_early_data_size** байтов открытого текста, и снимает с них защиту на ключе, построенном по **client_handshake_traffic_secret** (см. 9.4). Сервер отбрасывает записи, при снятии защиты с которых происходят ошибки. Первая запись, с которой успешно снята защита, считается началом второй очереди сообщений клиента. С этой записи сервер переходит на выполнение Handshake по схеме 1-RTT.

2 Сервер предлагает клиенту исправить **ClientHello**, отправляя сообщение **HelloRetryRequest**. Клиент не должен включать **early_data** в повторное сообщение **ClientHello**, отказываясь тем самым от 0-RTT. Сервер пропускает ранние прикладные данные клиента, отбрасывая записи **Record** с типом **application_data**. Это маскировочный тип, который означает, что записи защищены (см. 7.3). Сервер отбрасывает защищенные записи, которым в сумме соответствует не более **max_early_data_size** байтов открытого текста. Сервер находит запись с повторным сообщением **ClientHello** и с этого момента выполняет Handshake по схеме 2-RTT.

3 Сервер сообщает о поддержке 0-RTT, отправляя клиенту сообщение **EncryptedExtensions** с расширением **early_data**. Поддержка означает, что сервер будет принимать все ранние прикладные данные клиента. При этом в момент отправки **EncryptedExtensions** часть этих данных может быть в процессе доставки.

Прежде чем поддержать 0-RTT, сервер должен принять расширение **pre_shared_key** в сообщении **ClientHello**, обработать его по правилам, заданным в 6.3.13, и определить первый секрет, описанный в расширении.

После этого сервер должен проверить, что следующие параметры текущего соединения соответствуют атрибутам секрета:

- выбранная версия TLS;
- выбранный криптонабор;
- прикладной протокол, выбранный с помощью расширения **application_layer_protocol_negotiation** (см. таблицу 1), если таковой имеется.

Если какая-либо из проверок завершена с ошибкой, то сервер должен пропустить ранние прикладные данные клиента по правилам, заданным выше, переходя, таким образом, к 1-RTT или 2-RTT.

Дополнительно сервер анализирует описание секрета в расширении `pre_shared_key` клиента. Описание представляет собой структуру типа `PskIdentity` (см. 6.3.13). Если секрет был согласован через билет, отосланный сервером в сообщении `NewSessionTicket` предыдущего соединения, то поле `PskIdentity.obfuscated_ticket_age` содержит аддитивно зашумленное время жизни билета. Сервер проверяет время жизни, используя сохраненные реквизиты билета, в том числе величину зашумления, которая была отослана в поле `ticket_age_add` сообщения `NewSessionTicket` (см. 6.7.1). Сервер находит разность между `obfuscated_ticket_age` и `ticket_age_add` по модулю 2^{32} и проверяет, что результат незначительно отличается от времени в миллисекундах с момента выпуска билета. При нарушении этого условия серверу следует продолжить Handshake, но без поддержки 0-RTT. Кроме этого, серверу не следует полагаться на то, что сообщение `ClientHello` не просрочено.

Если сервер принял расширение `early_data`, то он должен обрабатывать записи с ранними прикладными данными по обычным правилам. В частности, при ошибке снятия защиты с записи сервер должен прервать Handshake с оповещением `bad_record_mac`.

При использовании механизма 0-RTT клиенту разрешается передавать ранние прикладные данные вплоть до сообщения `Finished` от сервера (см. 6.6). После получения `Finished` клиент отправляет сообщение `EndOfEarlyData` и стороны переходят к завершению Handshake. Чтобы избежать взаимной блокировки, сервер после приема `ClientHello` должен сразу же отправлять свою очередь сообщений, включая `ServerHello` с расширением `early_data`, не дожидаясь сообщения `EndOfEarlyData` от клиента.

Если сервер отклоняет `early_data`, то клиент может повторить отправку ранних прикладных данных, только теперь на обычной стадии — по завершении Handshake. Прикладной протокол может по-разному интерпретировать соединение до завершения Handshake и после завершения. Поэтому клиенту не следует автоматически, без участия прикладного протокола, повторно отправлять ранние прикладные данные. Клиент не должен автоматически повторно отправлять данные, если в процессе выполнения Handshake изменяется прикладной протокол, согласуемый через расширение `application_layer_protocol_negotiation`.

При определении расширений за пределами настоящего стандарта должны быть оговорены правила их применения относительно механизма 0-RTT.

6.3.13 Расширение `pre_shared_key`

Расширение `pre_shared_key` используется для согласования идентификатора секрета, который будет использоваться для формирования общих ключей в режимах PSK и PSK+DHE (см. 5.2).

Данные расширения `pre_shared_key` описываются типом `PreSharedKeyExtension`, который определяется следующим образом:

```
struct {
    opaque identity <1..216-1>;
    uint32 obfuscated_ticket_age;
} PskIdentity;

opaque PskBinderEntry <32..255>;
```

```
struct {
    PskIdentity identities<7.. $2^{16}-1$ >;
    PskBinderEntry binders<33.. $2^{16}-1$ >;
} OfferedPsk;
```

```
struct {
    select (Handshake.msg_type) {
        case client_hello: OfferedPsk;
        case server_hello:
            uint16 selected_identity;
    };
} PreSharedKeyExtension;
```

Клиент отправляет расширение `pre_shared_key` в сообщении `ClientHello`. Клиент указывает в расширении структуру типа `OfferedPsk`, поля которой имеют следующие значения:

- `identities` — список структур типа `PskIdentity`, которые описывают секреты. При включении в `ClientHello` расширения `early_data` первым в списке должно идти описание секрета для механизма 0-RTT (см. 6.3.12);
- `binders` — список PSK-скрепок. Каждый элемент `identities` должен сопровождаться скрепкой, порядок скрепок должен повторять порядок элементов. Правила вычисления скрепок определяются в 9.8.

Поля `PskIdentity` имеют следующее значение:

- `identity` — идентификатор секрета. В его качестве может выступать билет, высланный в сообщении `NewSessionTicket`, или имя, согласуемое за пределами TLS;
- `obfuscated_ticket_age` — зашумленное время жизни секрета. Если секрет согласован в предыдущем соединении через билет сообщения `NewSessionTicket`, то в поле должна быть указана сумма по модулю 2^{32} числа миллисекунд, прошедших с момента получения билета, и величины зашумления `ticket_age_add`, указанного в сообщении (см. 6.7.1). Клиент не должен использовать секрет, если с момента получения билета прошло больше `ticket_lifetime` секунд, где `ticket_lifetime` — величина, указанная в `NewSessionTicket`. Если секрет согласован за пределами TLS, то в поле `obfuscated_ticket_age` клиенту следует задавать значение 0, а сервер должен игнорировать поле.

Примечание — Зашумление времени жизни билета не позволяет пассивному противнику определить, в каком из предыдущих соединений билет был выпущен. Срок действия билета ограничен неделями и поэтому укладывается в 32-битовое поле `ticket_lifetime`.

Сервер отправляет расширение `pre_shared_key` в сообщении `ServerHello`. Сервер указывает в расширении номер выбранного секрета `selected_identity`. Это порядковый номер в списке `OfferedPsk.identities`, предложенном клиентом. Нумерация в списке начинается с 0.

Каждый секрет ассоциирован с определенным алгоритмом хэширования. Если секрет согласован в предыдущем соединении, то этим алгоритмом является алгоритм хэширования криптонабора, который использовался в соединении. Если секрет согласовывается за пределами TLS, то алгоритм хэширования должен определяться в момент согласования. При выборе секрета сервер должен обеспечить его совместимость с криптонабором текущего соединения: алгоритм хэширования криптонабора должен совпадать с алгоритмом хэширования, который ассоциирован с секретом.

Перед выбором секрета сервер должен проверить соответствующую PSK-скрепку. Правила проверки определены в 9.8. Если скрепка отсутствует или оказывается некорректной, то сервер должен прервать Handshake. Серверу не следует проверять сразу несколько скрепок — следует выбрать единственный секрет и проверить только его скрепку.

Получив сообщение `ServerHello`, клиент находит расширение `pre_shared_key` и проверяет, что указанный в нем номер `selected_identity` не выходит за границы списка `OfferedPskIdentities`, предложенного клиентом. Кроме этого, клиент проверяет, что выбранные сервером криптонабор и секрет совместимы друг с другом. Наконец, клиент проверяет, что `ServerHello` содержит расширение `key_share`, если этого требовало расширение `psk_key_exchange_modes`, отправленное клиентом в `ClientHello`. При нарушении любого из условий клиент должен прервать Handshake с оповещением `illegal_parameter`.

Если сервер включил в `ServerHello` расширение `early_data`, то клиент должен убедиться, что выбранный сервером номер `selected_identity` равняется 0. При нарушении этого условия клиент должен прервать Handshake с оповещением `illegal_parameter`.

Чтобы упростить построение PSK-скрепок, расширение `pre_shared_key` должно идти последним в списке расширений `ClientHello`. Сервер должен проверять данное условие и прерывать Handshake с оповещением `illegal_parameter` в случае его нарушения.

6.4 Параметры сервера

6.4.1 Сообщение EncryptedExtensions

В сообщении `EncryptedExtensions` сервер пересылает защищенные расширения. Формат сообщения описывается одноименным типом:

```
struct {
    Extension extensions<0..216-1>;
} EncryptedExtensions;
```

В поле `extensions` указывается список расширений. Допустимые расширения перечислены в таблице 1.

Сервер должен выслать `EncryptedExtensions` сразу после своего сообщения `ServerHello`.

Сообщение `EncryptedExtensions` защищается на ключе, построенном по `server_handshake_traffic_secret` (см. 9.4). Это первое сообщение, которое защищается на данном ключе.

Клиент должен проверить, что `EncryptedExtensions` содержит только допустимые расширения. При нарушении условия клиент должен прервать Handshake с оповещением `illegal_parameter`.

6.4.2 Сообщение CertificateRequest

С помощью сообщения `CertificateRequest` сервер запрашивает сертификат клиента для его аутентификации. Сервер может выслать `CertificateRequest`, только если сам аутентифицируется по сертификату.

Формат сообщения `CertificateRequest` описывается одноименным типом:

```
struct {
    opaque certificate_request_context<0..28-1>;
    Extension extensions<0..216-1>;
} CertificateRequest;
```

Поля `CertificateRequest` имеют следующее значение:

- `certificate_request_context` — строка, которая идентифицирует запрос сертификата и которая будет дублироваться в сообщении `Certificate` клиента. Если в соединении высылается несколько сообщений `CertificateRequest`, то их строки `certificate_request_context` не должны повторяться. Если `CertificateRequest` высылается в Handshake, а не по его завершении, то строка должна иметь нулевую длину. При отправке `CertificateRequest` после Handshake серверу следует выбирать строки так, чтобы клиент не мог их предсказать (например, генерируя случайным образом);

- `extensions` — перечень расширений, которые описывают параметры запрашиваемого сертификата. Перечень должен содержать расширение `signature_algorithms`, может содержать другие расширения. Клиент должен игнорировать нераспознанные расширения.

Сообщение `CertificateRequest` не является обязательным. Если сервер высылает `CertificateRequest`, то он должен это сделать сразу после `EncryptedExtensions`.

Если сервер аутентифицируется с помощью PSK, согласованного в предыдущем соединении, то он не должен отправлять `CertificateRequest` в протоколе Handshake, хотя может это сделать по завершении протокола (см. 6.7.2) при условии, что клиент выслал в `ClientHello` расширение `post_handshake_auth` (см. 6.3.8). Если сервер аутентифицируется с помощью PSK, согласованного за пределами TLS, то он не должен отправлять `CertificateRequest` ни в протоколе Handshake, ни по его завершении.

Сообщение `CertificateRequest` защищается на ключе, построенном по `server_handshake_traffic_secret` (см. 9.4).

6.5 Аутентификация

6.5.1 Сообщение Certificate

В сообщении `Certificate` сторона передает свой сертификат для аутентификации, а также другие сертификаты и данные, которые подтверждают его подлинность.

Формат сообщения `Certificate` описывается одноименным типом:

```
enum {
    X509(0),
    RawPublicKey(2),
    (255)
} CertificateType;

struct {
    select (certificate_type) {
        case RawPublicKey:
            opaque ASN1_subjectPublicKeyInfo<1..2^24-1>;
        case X509:
            opaque cert_data<1..2^24-1>;
    };
    Extension extensions<0..2^16-1>;
} CertificateEntry;

struct {
    opaque certificate_request_context<0..2^8-1>;
    CertificateEntry certificate_list<0..2^24-1>;
}
```

```
} Certificate;
```

Поля `Certificate` имеют следующее значение:

- `certificate_request_context` — синхропосылка. Если сообщение является ответом на запрос `CertificateRequest`, то в поле должна быть повторена синхропосылка запроса. Иначе поле должно быть пустым (иметь нулевую длину);
- `certificate_list` — список (цепочка) структур типа `CertificateEntry`, каждая из которых содержит один сертификат и сопровождающий его набор расширений.

Если стороны не согласовывали тип сертификата с помощью расширения `server_certificate_type` или `client_certificate_type` или если согласован тип `CertificateType.X509`, то элементы списка `certificate_list` должны содержать сертификаты X.509 (см. СТБ 34.101.19). Сертификат кодируется по правилам DER, код размещается в поле `CertificateEntry.cert_data`. Сертификат отправителя сообщения `Certificate` должен быть первым в цепочке `certificate_list`. Следующие сертификаты следует располагать так, чтобы каждый из них удостоверял предыдущий. Поскольку точки доверия (корневые сертификаты) согласуются за пределами TLS, они могут быть исключены из цепочек при условии, что стороны знают, как их восстановить.

Если согласован тип сертификата `CertificateType.RawPublicKey`, то список `certificate_list` должен содержать не более одного элемента `CertificateEntry`. Если список непуст, то в поле `ASN1_subjectPublicKeyInfo` его единственного элемента размещается DER-код контейнера `SubjectPublicKeyInfo` (см. 6.3.1).

Примечание — Контейнер `SubjectPublicKeyInfo` содержит открытый ключ, а не удостоверяющую подпись, обычную для сертификатов. Тем не менее, контейнер выполняет функции сертификата, если сторона TLS ведет список доверенных контейнеров, согласованный за пределами TLS, и проверяет, что присланный в `Certificate` контейнер входит в список.

Сопровождающие сертификат расширения задаются в поле `CertificateEntry.extensions`. Они описываются типом `Extension`, который определен в 6.3.1. Расширения в сообщении `Certificate` сервера должны соответствовать расширениям в `ClientHello`, а расширения в `Certificate` клиента — расширениям в `CertificateRequest`. Если расширение относится ко всей цепочке сертификатов, то оно должно быть указано в первом элементе списка `certificate_list`. Настоящий стандарт разрешает использовать только расширение `status_request` (см. 6.3.7). Перечень допустимых расширений может дополняться за пределами стандарта.

Сервер должен отправить сообщение `Certificate`, если общие ключи были сформированы в режиме DHE, т. е. без использования предварительно согласованных секретов. Список сертификатов сервера должен быть непустым.

Клиент должен отправить сообщение `Certificate`, только и только если сервер запрашивает аутентификацию с помощью сообщения `CertificateRequest`. Если сервер запрашивает аутентификацию, а у клиента отсутствует подходящий сертификат, то клиент должен отправить сообщение `Certificate` с пустым списком `certificate_list`, т. е. без сертификатов.

Сообщение `Certificate` защищается на ключе, построенном по `client_handshake_traffic_key` или `server_handshake_traffic_key` в зависимости от отправителя (см. 9.4).

6.5.2 Выбор и проверка сертификатов

Клиент и сервер должны использовать сертификаты X.509 (см. СТБ 34.101.19), если иной тип не согласован с помощью расширений `client_certificate_type` и `server_certificate_type` (см. таблицу 1).

Открытый ключ сертификата должен быть совместим с одним из алгоритмов ЭЦП, указанных противоположной стороной в расширении `signature_algorithms` (см. 6.3.4).

Примечание — Возможна ситуация, когда открытый ключ сертификата используется с одним алгоритмом ЭЦП, а сам сертификат подписан другим алгоритмом.

Дополнительные требования к сертификатам X.509:

- в сертификат должно быть включено расширение `KeyUsage`, и в нем должен быть установлен флаг `digitalSignature`;
- если противоположная сторона выслала расширение `certificate_authorities`, то в цепочку сертификатов `Certificate` следует включить хотя бы один сертификат, который удовлетворяет требованиям расширения (см. 6.3.5);
- если сервер включил в `CertificateRequest` расширение `oid_filters`, то сертификат клиента должен удовлетворять требованиям расширения (см. 6.3.6);
- если сервер может использовать расширение `server_name` (см. таблицу 1) при выборе своего сертификата, то клиенту следует высылать это расширение.

Все сертификаты X.509, включенные в `Certificate`, должны быть подписаны с помощью алгоритмов, указанных в расширениях `signature_algorithms` и `signature_algorithms_cert` противоположной стороны. Требование не распространяется на точки доверия, подписи которых могут не проверяться.

Если отправителем `Certificate` является сервер и он не может подготовить цепочку сертификатов, в которой используются только запрошенные алгоритмы ЭЦП, то серверу следует продолжить Handshake, составив резервную цепочку по своему усмотрению. В резервную цепочку могут входить сертификаты, для алгоритмов проверки подписей которых неизвестно, поддерживает ли их клиент или нет.

Если отправителем `Certificate` является клиент и он также не может выполнить ограничения на алгоритмы ЭЦП, то он может либо выслать резервную цепочку, либо продолжить Handshake анонимно, т. е. с пустым списком сертификатов в `Certificate`.

Если получатель `Certificate` не может по присланному списку сертификатов построить подходящую цепочку, то он должен прервать Handshake с подходящим оповещением (`unsupported_certificate` по умолчанию).

Если у отправителя `Certificate` есть несколько подходящих сертификатов, то он может выбрать любой из них, руководствуясь изложенными правилами и дополнительными критериями.

Цепочка сертификатов X.509 должна проверяться в соответствии с СТБ 34.101.19. Если проверку выполняет сервер, то в случае ошибки он может по своему усмотрению либо продолжить Handshake без аутентификации клиента, либо прервать его.

Если сервер выслал в `Certificate` пустой список сертификатов, то клиент должен прервать Handshake с оповещением `decode_error`.

Если пустой список сертификатов выслал клиент, то сервер может по своему усмотрению либо продолжить Handshake без аутентификации клиента, либо прервать его с оповещением `certificate_required`.

6.5.3 Сообщение CertificateVerify

Сообщение `CertificateVerify` содержит подпись текущей стенограммы `Handshake`. С помощью `CertificateVerify` отправитель доказывает, что он действительно владеет личным ключом, связанным с его сертификатом. Кроме этого, подпись в `CertificateVerify` подтверждает целостность и подлинность стенограммы.

Формат сообщения `CertificateVerify` описывается одноименным типом:

```
struct {
    SignatureScheme algorithm;
    opaque signature<0..216-1>;
} CertificateVerify;
```

Поля `CertificateVerify` имеют следующее значение:

- `algorithm` — идентификатор алгоритмов ЭЦП. Описывается типом `SignatureScheme`, который определен в 6.3.4;
- `signature` — подпись стенограммы `Handshake`.

Подпись вырабатывается по правилам, заданным в 9.7. При выработке подписи используется личный ключ отправителя сообщения.

Сервер должен отправлять `CertificateVerify` при аутентификации по сертификату после сообщения `Certificate`. Клиент должен отправлять `CertificateVerify` при аутентификации по сертификату, если сертификат указан в `Certificate` (список сертификатов непуст).

Сообщение `CertificateVerify` должно отправляться непосредственно после `Certificate` и непосредственно перед `Finished`.

Идентификатор алгоритмов ЭЦП в сообщении `CertificateVerify` сервера должен входить в список, указанный клиентом в расширении `signature_algorithms` сообщения `ClientHello`. Идентификатор алгоритмов ЭЦП в `CertificateVerify` клиента должен входить в список, указанный сервером в расширении `signature_algorithms` сообщения `CertificateRequest`. Открытый ключ в сертификате отправителя `CertificateVerify` должен соответствовать алгоритмам ЭЦП.

Получатель `CertificateVerify` должен проверить подпись в поле `signature`. При проверке должен использоваться открытый ключ из сертификата отправителя, присланного в сообщении `Certificate`. Если проверка подписи завершена с ошибкой, то получатель должен прервать `Handshake` с оповещением `decrypt_error`.

Сообщение `CertificateVerify` защищается на ключе, построенном по `client_handshake_traffic_key` или `server_handshake_traffic_key` в зависимости от отправителя.

6.5.4 Сообщение Finished

В сообщении `Finished` пересылается имитовставка текущей стенограммы `Handshake`. Имитовставка вычисляется на одном из общих ключей. Ее корректность подтверждает знание общих ключей, целостность и подлинность стенограммы.

Формат сообщения `Finished` определяется одноименным типом:

```
struct {
    opaque verify_data[hash_length];
} Finished;
```

В поле `verify_data` указывается имитовставка стенограммы. Имитовставка вычисляется по правилам, заданным в 9.7.

Получатель **Finished** должен проверить имитовставку. В случае ошибки получатель должен прервать Handshake с оповещением **decrypt_error**.

После того, как сервер отправил свое сообщение **Finished**, получил и проверил ответное **Finished**, он может начать отправку прикладных данных. Сервер может отправлять прикладные данные даже раньше, не дожидаясь **Finished** клиента. Однако при этом в момент отправки у сервера нет гарантий, что клиент действительно выполняет протокол, а не просто выслал **ClientHello**. Клиент может отправлять прикладные данные, не дожидаясь **Finished** сервера, в рамках механизма 0-RTT.

Сообщение **Finished** защищается на ключе, построенном по **client_handshake_traffic_key** или **server_handshake_traffic_key** в зависимости от отправителя.

После **Finished** отправляемые данные должны защищаться на ключе, построенном по **client_application_traffic_key** или **server_application_traffic_key**. В частности, оповещения **Alert**, которые могут появиться при обработке сообщений **Certificate** и **CertificateVerify** клиента, сервер защищает на ключе, построенном по **server_application_traffic_key**.

6.6 Сообщение EndOfEarlyData

Сообщение **EndOfEarlyData** информирует о завершении передачи ранних прикладных данных в рамках механизма 0-RTT.

Формат сообщения **EndOfEarlyData** описывается одноименным типом:

```
struct {} EndOfEarlyData;
```

Если сервер согласился с использованием механизма 0-RTT, отправив расширение **early_data** в сообщении **EncryptedExtensions**, то клиент должен отправить **EndOfEarlyData** после приема сообщения **Finished** от сервера. Если сервер не включил **early_data** в **EncryptedExtensions**, то клиент не должен высылать **EndOfEarlyData**.

Сообщение **EndOfEarlyData** защищается на ключе, построенном по **client_early_traffic_secret** (см. 9.4). После отправки **EndOfEarlyData** клиент защищает свои сообщения на ключе, построенном по **client_handshake_traffic_secret**.

Сервер не должен отправлять сообщение **EndOfEarlyData**. Если клиент получает данное сообщение, то он должен закрыть соединение с оповещением **unexpected_message**.

6.7 Сообщения после Handshake

6.7.1 Сообщение NewSessionTicket

В сообщении **NewSessionTicket** сервер высылает клиенту билет, который ссылается на секрет, построенный по общему ключу **resumption_master_secret** (см. 9.4). Используя **resumption_master_secret** и данные билета, клиент может построить секрет самостоятельно и использовать его в будущих соединениях в качестве PSK.

В одном соединении сервер может отправить клиенту несколько билетов: последовательно друг за другом либо после определенных событий. Например, сервер может отправлять билет после аутентификации клиента по завершении Handshake, фиксируя факт аутентификации. Располагая несколькими билетами, клиент может открывать несколько параллельных соединений с сервером.

Сервер может отправить сообщение **NewSessionTicket** в любой момент после получения сообщения **Finished** клиента при условии, что **ClientHello** содержит расширение **psk_key_exchange_mode** (см. 6.3.11).

Сообщение `NewSessionTicket` защищается на ключе, построенном по `server_application_traffic_secret` (см. 9.4).

Клиент может использовать PSK, на который ссылается билет в `NewSessionTicket`, для возобновления связи. Для этого клиент должен включить билет в качестве идентификатора PSK в расширение `pre_shared_key` (см. 6.3.13). Если клиент не поддерживает возобновление связи, то он должен игнорировать сообщение `NewSessionTicket`.

При возобновлении связи сервер должен принимать билет, только если он был построен с помощью алгоритма хэширования текущего криптонабора.

При использовании расширения `server_name` клиенту не следует возобновлять связь, если билет был согласован в соединении с именем сервера, которое отличается от текущего имени. Однако клиент может возобновить связь, если он был проинформирован об изменении имени сервера за пределами TLS. Клиент не должен возобновлять связь, если текущее имя сервера не соответствует данным в сертификате сервера, который был использован в первоначальном соединении. Если при возобновлении связи требуется сообщить имя сервера прикладному протоколу, то следует указать текущее имя, а не имя в первоначальном соединении.

Клиенту не следует использовать один и тот же билет в нескольких соединениях, поскольку повтор билета позволяет сопоставить соединения друг с другом. Если сервер выпускает билеты, то их число должно быть не меньше, чем планируемое количество соединений с клиентом. Серверу следует выпускать билеты в каждом соединении. Это дает клиенту возможность использовать свежий билет всякий раз при открытии нового соединения.

Формат сообщения `NewSessionTicket` определяется одноименным типом:

```
struct {
    uint32 ticket_lifetime;
    uint32 ticket_age_add;
    opaque ticket_nonce<0..255>;
    opaque ticket<1..216-1>;
    Extension extensions<0..216-2>;
} NewSessionTicket;
```

Поля `NewSessionTicket` имеют следующее значение:

- `ticket_lifetime` – срок действия билета в секундах, начиная с момента его выпуска. Срок действия не должен быть больше 604 800 секунд (7 суток). Нулевое значение `ticket_lifetime` означает, что билет следует уничтожить немедленно. Клиент не должен использовать билет по прошествии 7 суток после его выпуска независимо от значения, установленного в `ticket_lifetime`. Срок действия билета может сокращаться относительно `ticket_lifetime` в соответствии с локальной политикой пользователя. Сервер может аннулировать билет до окончания срока его действия;
- `ticket_age_add` — случайное 32-битовое число, которое используется для зашумления времени жизни билета. Клиент получает зашумленное время жизни, складывая актуальное время в миллисекундах с `ticket_age_add` по модулю 2^{32} . Клиент высылает зашумленное время в расширении `pre_shared_key` (см. 6.3.13). При формировании очередного билета сервер должен генерировать `ticket_age_add` заново;
- `ticket_nonce` — синхропосылка. Синхропосылки текущего соединения не должны повторяться;

- **ticket** — билет. Выступает в роли идентификатора ассоциированного PSK. Билетом может быть ключ базы данных или (аутентифицированный) шифртекст, который проверяется при получении;
- **extensions** — перечень расширений. Клиент должен игнорировать нераспознанные расширения.

Настоящий стандарт разрешает включать в перечень `NewSessionTicket.extensions` только расширение `early_data` (см. 6.3.12). Включая расширение, сервер объявляет, что секрет, на который ссылается выпущенный билет, может использоваться в механизме 0-RTT.

В поле `max_early_data_size` расширения `early_data` сервер указывает, сколько байтов ранних прикладных данных разрешается передать клиенту в рамках механизма 0-RTT. Если сервер получает более `max_early_data_size` байтов ранних прикладных данных, то ему следует закрыть соединение с оповещением `unexpected_message`.

Лимит `max_early_data_size` не учитывает однобайтовый тип содержимого и незначащие нулевые байты в `TLSInnerPlaintext` (см. 7.3). Сервер может проверять соблюдение лимита, не снимая с данных защиту, и поэтому клиенту не следует дополнять записи `Record` большим числом незначащих нулей.

Секрет, связанный с билетом, вычисляется по правилам, установленным в 9.9.2.

Примечания

1 Если сервер не запрашивает аутентификацию клиента по сертификату, то он может выслать билет в `NewSessionTicket` сразу после своего сообщения `Finished`. Для этого сервер самостоятельно вычисляет сообщение `Finished` клиента, определяет полную стенограмму `Handshake`, строит по ней сначала ключ `resumption_master_secret`, а затем билет.

2 Сформировав один раз общие ключи, сервер может продлевать время их жизни, последовательно выпуская новые и новые билеты. Рекомендуются контролировать и ограничивать время жизни такого ключевого материала. В частности, если первоначальные ключи построены с использованием сертификатов, то при контроле следует учитывать срок действия сертификатов, возможность их отзыва.

6.7.2 Отложенная аутентификация

Если клиент отправил расширение `post_handshake_auth` (см. 6.3.8), то он может быть аутентифицирован не только в протоколе `Handshake`, но и по его завершении.

Сервер запрашивает отложенную аутентификацию, отправляя сообщение `CertificateRequest`.

Клиент должен ответить сообщениями аутентификации (см. 6.5). Если клиент согласен аутентифицироваться, то он должен выслать сообщения `Certificate`, `CertificateVerify` и `Finished`. Если клиент отказывается, то он должен выслать сообщение `Certificate`, которое не содержит сертификатов, а после него сообщение `Finished`. Сообщения должны идти в оговоренном порядке и не должны перемежаться сообщениями других типов или от других запросов.

Сообщение `CertificateRequest` и ответные сообщения клиента защищаются на ключах, построенных по `server_application_traffic_secret` и `client_application_traffic_secret` соответственно (см. 9.4).

Если клиент не отправлял расширение `post_handshake_auth`, но получил `CertificateRequest` после `Handshake`, то он должен закрыть соединение с оповещением `unexpected_message`.

Поскольку аутентификация клиента может включать запрос пользователю (владельцу личного ключа), сервер должен быть готов к задержке между `CertificateRequest` и

Finished. Во время задержки сервер может получать сообщения, не относящиеся к аутентификации. Кроме этого, порядок ответов клиента на несколько подряд идущих запросов **CertificateRequest** не обязательно совпадает с порядком запросов. Синхропосылка **certificate_request_context** в сообщениях **CertificateRequest** и **Certificate** помогает серверу связать запросы с ответами.

6.7.3 Сообщение KeyUpdate

Сообщение **KeyUpdate** информирует об обновлении ключа защиты отправляемых прикладных данных. Правила обновления определены в 9.5.

Формат сообщения **KeyUpdate** описывается одноименным типом:

```
enum {
    update_not_requested(0), update_requested(1), (255)
} KeyUpdateRequest;

struct {
    KeyUpdateRequest request_update;
} KeyUpdate;
```

В поле **request_update** отправитель сообщения информирует о необходимости отправки встречного **KeyUpdate** или, другими словами, о необходимости обновления ключа защиты принимаемых прикладных данных. Значение **update_requested** означает необходимость отправки встречного сообщения, значение **update_not_requested** — отсутствие необходимости. Получатель должен закрыть соединение с оповещением **unexpected_message**, если в поле указано недопустимое значение.

Сообщение **KeyUpdate** может отправить любая сторона после того, как она выслала сообщение **Finished** и завершила, таким образом, **Handshake**. Если клиент или сервер получает **KeyUpdate** перед **Finished**, то он должен закрыть соединение с оповещением **unexpected_message**.

После передачи **KeyUpdate** отправитель должен использовать обновленный ключ защиты отправляемых прикладных данных. При этом само сообщение **KeyUpdate** защищается на необновленном ключе. Стороны не должны принимать записи, защищенные на обновленном ключе, вплоть до получения **KeyUpdate**, защищенного на необновленном. Ключи защиты строятся по **server_application_traffic_secret** или **client_application_traffic_secret**, в зависимости от того, кто является отправителем (см. 9.6).

Если в поле **request_update** сообщения **KeyUpdate** указано значение **update_requested**, то получатель должен ответить своим собственным **KeyUpdate**, прежде чем отправлять прикладные данные. В поле **request_update** встречного **KeyUpdate** должно быть указано значение **update_not_requested**. Допустима ситуация, когда сторона получает несколько **KeyUpdate** с запросом на встречное сообщение и отвечает только одним таким сообщением. Сообщение **KeyUpdate** с запросом запрещено отправлять повторно до получения искомого встречного **KeyUpdate**.

Примечания

1 После отправки **KeyUpdate** с запросом на встречное сообщение, сторона может получить это сообщение не сразу, а после других сообщений, которые находятся в процессе доставки. При этом в процессе доставки могут находиться запаздывающие сообщения **KeyUpdate**, не связанные с отправленным.

2 Если стороны независимо одновременно отправляют друг другу сообщения **KeyUpdate** с запросами на встречные сообщения, то, ответив на запросы, каждая сторона в конце концов обновит ключ отправляемых ею данных дважды.

7 Протокол Record

7.1 Записи

Протокол Record предназначен для передачи данных. Данные разбиваются на фрагменты, фрагменты помещаются в записи, записи пересылаются противоположной стороне. При получении записей их фрагменты объединяются и передаются протоколам верхних уровней. Записи могут пересылаться в защищенной форме.

В записи указывается ее тип. Различные протоколы верхних уровней могут передавать свои сообщения через общий протокол Record, используя записи различных типов.

В настоящем стандарте определены четыре типа записей:

- **handshake** — сообщения протокола Handshake;
- **application_data** — сообщения прикладных протоколов;
- **alert** — сообщения протокола Alert;
- **change_cipher_spec** — используется для совместимости с промежуточными сетевыми устройствами, ориентированными на работу с сообщениями TLS 1.2.

Реализации TLS не должны отправлять записи, типы которых отличаются от перечисленных выше. Исключение могут составить типы, которые будут устанавливаться для расширений TLS за рамками настоящего стандарта. Если реализация получает запись неустановленного типа, то она должна прервать протокол с оповещением **unexpected_message**.

Запись типа **change_cipher_spec** может быть получена в процессе выполнения Handshake после отправки или получения первого сообщения **ClientHello** (отличного от **ClientHello**, которое следует после **HelloRetryRequest**) и до того, как получено сообщение **Finished** от противоположной стороны. Запись должна быть незащищенной и содержать однобайтовый фрагмент 0x01. При соблюдении данных условий полученная запись должна игнорироваться. При нарушении любого из условий протокол Handshake должен быть прерван с оповещением **unexpected_message**. Протокол должен быть прерван с тем же оповещением, если обнаружено, что запись **change_cipher_spec** получена до первого **ClientHello** или после ответного **Finished**. Сервер без состояния, лишенный возможности обнаруживать нарушение порядка записей, может игнорировать последнее требование.

7.2 Фрагментация

Запись содержит фрагмент данных из 2^{14} или менее байтов. Длины фрагментов формируются с учетом типа записей, т. е. протокола верхнего уровня, который использует Record для передачи своих сообщений. Определения новых типов следует сопровождать соглашениями о длинах.

Сообщения Handshake могут передаваться в одной или нескольких записях. Должны соблюдаться следующие ограничения:

- если сообщение Handshake передается в двух или более записях, то между этими записями не должно быть записей других типов;
- сообщение, которое непосредственно предшествует изменению ключей, должно заканчиваться на границе записи. Это условие должно проверяться при приеме записей. При нарушении условия соединение должно быть разорвано с оповещением **unexpected_message**. Ограничение распространяется на сообщения **ClientHello**,

`EndOfEarlyData`, `ServerHello`, `Finished` и `KeyUpdate`, которые могут непосредственно предшествовать изменению ключей;

- фрагмент сообщения `Handshake` не должен иметь нулевую длину, даже если он сопровождается дополнением.

Сообщения протокола `Alert` не должны передаваться в нескольких записях, и, наоборот, в одной записи не должно передаваться несколько сообщений `Alert`. Другими словами, запись `Alert` должна содержать ровно одно сообщение.

Данные прикладных протоколов непрозрачны для TLS. Данные могут передаваться в одной или нескольких записях. Записи прикладных протоколов всегда защищены. Записи могут содержать фрагменты нулевой длины, что позволяет защищаться от анализа трафика (см. 7.5).

Записи TLS описываются типом `TLSPlaintext`, который определяется следующим образом:

```
enum {
    invalid(0),
    change_cipher_spec(20),
    alert(21),
    handshake(22),
    application_data(23),
    (255)
} ContentType;

struct {
    ContentType type;
    ProtocolVersion legacy_record_version;
    uint16 length;
    opaque fragment[TLSPlaintext.length];
} TLSPlaintext;
```

Поля `TLSPlaintext` имеют следующее значение:

- `type` — тип записи (см. 7.1);
- `legacy_record_version` — устаревшее поле, которое должно игнорироваться после получения и проверки. Если в записи передается первое сообщение `ClientHello` протокола `Handshake`, то в поле может указываться номер `0x0301`. Во всех остальных случаях в поле должен быть установлен номер `0x0303`.

Примечание — Номер `0x0301` соответствует TLS 1.0, номер `0x0303` — TLS 1.2. Установка номера `0x0301` в записи для первого `ClientHello` и номера `0x0303` в записи для повторного `ClientHello` или `ServerHello` обеспечивает максимальную совместимость с предыдущими версиями TLS;

- `length` — длина (в байтах) поля `TLSPlaintext.fragment`. Длина не должна быть больше 2^{14} . При получении записи с большей длиной протокол должен быть прерван с оповещением `record_overflow`;

- `fragment` — фрагмент данных. Обрабатывается протоколом верхнего уровня, который определяется по полю `type`.

Структуры `TLSPlaintext` пересылаются между сторонами, если не включена защита записей. Если защита включена, то структуры защищаются перед отправкой. Правила защиты определяются в 7.3.

7.3 Защита записей

При защите записей структура `TLSPlaintext` преобразуется в структуру `TLSCiphertext`. При снятии защиты выполняется обратное преобразование. Для защиты используются алгоритмы аутентифицированного шифрования с ассоциированными данными (Authenticated Encryption with Associated Data (AEAD)). Алгоритм установки защиты принимает на вход открытый текст, открытые ассоциированные данные, ключ и синхропосылку и возвращает аутентифицированный шифртекст. В качестве последнего обычно выступают собственно шифртекст и сопровождающая его имитовставка. Алгоритм снятия защиты принимает на вход аутентифицированный шифртекст, открытые ассоциированные данные, ключ и синхропосылку и возвращает либо открытый текст, либо признак ошибки. Ошибка означает нарушение целостности входных данных.

Аутентифицированный шифртекст обычно имеет больший объем, чем соответствующий открытый текст. Должны использоваться такие алгоритмы аутентифицированного шифрования, которые расширяют открытый текст не более чем на 255 байт.

При установке защиты структуры `TLSPlaintext` открытый текст оформляется в виде структуры типа `TLSInnerPlaintext`. Этот тип определяется следующим образом:

```
struct {
    opaque content[TLSPlaintext.length];
    ContentType type;
    uint8 zeros[length_of_padding];
} TLSInnerPlaintext;
```

Поля `TLSInnerPlaintext` имеют следующее значение:

- `content` — `TLSPlaintext.fragment`, фрагмент защищаемой записи;
- `type` — `TLSPlaintext.type`, тип защищаемой записи;
- `zeros` — нулевые байты, которые маскируют `TLSPlaintext.length`. Разрешается дописывать произвольное число нулевых байтов при условии, что размер структуры `TLSInnerPlaintext` остается в пределах ограничений, установленных в 7.5.

Защищенная запись описывается типом `TLSCiphertext`, который определяется следующим образом:

```
struct {
    ContentType opaque_type = application_data;
    ProtocolVersion legacy_record_version = 0x0303;
    uint16 length;
    opaque encrypted_record[TLSCiphertext.length];
} TLSCiphertext;
```

Поля `TLSCiphertext` имеют следующее значение:

- `opaque_type` — маскировочный тип, который всегда устанавливается равным `application_data` (23). Выбранное значение обеспечивает максимальную совместимость с промежуточными сетевыми устройствами, ориентированными на работу с предыдущими версиями TLS. Фактический тип защищенной записи определяется по полю `TLSInnerPlaintext.type` после снятия защиты;

- `legacy_record_version` — устаревшее поле, в котором должен быть задан номер 0x0303.

Примечание — В поле указывается версия записей TLS. Поскольку записи TLS версии 1.3 могут пересылаться только после того, как данная версия согласована в Handshake, пробле-

мы с совместимостью не возникают. Поскольку версия TLS согласована в Handshake, поле является избыточным;

- **length** — длина в байтах поля `TLSCiphertext.encrypted_record`, следующего за данным. Длина представляет собой сумму длины открытого текста (включая содержимое, тип и нулевые байты дополнения) и длины его расширения при аутентифицированном шифровании. Поле **length** не должно превышать $2^{14} + 256$. При нарушении этого условия принимающая сторона должна разорвать соединение с оповещением **record_overflow**;
- **encrypted_record** — шифртекст, полученный в результате защиты `TLSPinnerPlaintext`.

При установке защиты `TLSPinnerPlaintext` в качестве открытого текста выступает конкатенация полей **content**, **type** и **zeros**. Ассоциированными данными является конкатенация трех полей `TLSCiphertext`: **opaque_type**, **legacy_record_version** и **length**. Используется ключ **client_write_key** или **server_write_key**, который определяется по правилам, заданным в 9.6. Если отправителем записи является клиент, то используется ключ **client_write_key**, если сервер — ключ **server_write_key**. Синхропосылка строится по правилам, заданным в 7.4.

Защита снимается с `TLSCiphertext.encrypted_record` с теми же ассоциированными данными, ключом и синхропосылкой. В случае ошибки при снятии защиты соединение должно быть разорвано с оповещением **bad_record_mac**.

7.4 Синхропосылки

При передаче и приеме записей используются независимые 64-битовые порядковые номера. Номер должен устанавливаться равным 0 в самом начале обмена данными и после смены ключей. Номер должен увеличиваться на 1 после обработки (передачи или приема) каждой записи.

Число всевозможных порядковых номеров, 2^{64} , весьма велико и поэтому при обработке записей номер не переполнится. Для принудительного сброса номера следует обновить ключи (см. 6.7.3) или разорвать соединение.

Алгоритмы аутентифицированного шифрования, которые используются для защиты записей, характеризуются минимальной и максимальной длиной синхропосылок в байтах. Алгоритмы, у которых максимальная длина меньше 8, не должны использоваться в TLS. Длина синхропосылки для защиты записи, **iv_length**, должна выбираться как максимум из минимальной длины и 8.

Синхропосылка формируется следующим образом:

- 1 Порядковый номер записи кодируется строкой байтов по правилу «от старших к младшим» (big-endian, сетевой порядок байтов), а затем дополняется слева нулями до **iv_length** байтов.
- 2 Полученная строка складывается поразрядно по модулю 2 с начальной синхропосылкой **client_write_iv** или **server_write_iv**, в зависимости от того, кто является отправителем — клиент или сервер. Начальная синхропосылка формируется по правилам, заданным в 9.6.
- 3 Результат сложения объявляется искомой синхропосылкой.

7.5 Дополнение записей

Записи TLS перед установкой защиты могут дополняться незначащими байтами, маскируя тем самым длину пересылаемых фрагментов данных и затрудняя анализ трафика.

Например, клиент и сервер могут скрывать отсутствие активности в прикладном протоколе, обмениваясь пустыми фрагментами, но дополняя их.

Реализации TLS могут самостоятельно выбирать политику дополнения, включая условия установки дополнения и число дополнительных нулей. Дополнение не является обязательным механизмом.

Дополнительные байты записываются в поле **zeros** структуры **TLSTInnerPlaintext**. Дополнительные байты должны быть нулевыми.

Дополнение снимается после снятия защиты с **TLSTInnerPlaintext**. Учитывается, что перед **zeros** размещено ненулевое поле **type**. Байты **TLSTInnerPlaintext** просматриваются от конца к началу. Нулевые байты отбрасываются до тех пор, пока не встретится ненулевой. Если ненулевой байт не найден, то соединение должно быть разорвано с оповещением **unexpected_message**.

Размер структуры **TLSTInnerPlaintext** не должен превышать $2^{14} + 1$ байт. Размер должен определяться с учетом дополнения. Если максимальный размер **TLSTInnerPlaintext** уменьшается, например, через расширение **max_fragment_length** (см. таблицу 1), то ограничение на размер должно продолжать учитывать дополнение.

7.6 Квоты ключей

Алгоритмы аутентифицированного шифрования остаются надежными до тех пор, пока соблюдаются квоты для используемых в них ключей. Квота ключа — это максимальный объем данных, которые разрешается обработать без смены ключа. Пример квот представлен в СТБ 34.101.31 (приложение В).

Стороны TLS должны обновлять ключи при достижении квот.

8 Протокол Alert

8.1 Оповещения

Протокол Alert предназначен для информирования о закрытии соединения и об ошибках. Сообщения Alert, называемые оповещениями, защищаются, как и другие сообщения TLS, в соответствии с текущим состоянием соединения. Оповещения о закрытии соединения считаются предупредительными, оповещения об ошибках — критическими.

При получении критического оповещения стороне TLS следует проинформировать об ошибке прикладной протокол и уничтожить PSK, связанные с билетами сеансов (session tickets). Остальные секретные данные, сформированные в соединении, должны быть уничтожены обязательно. Кроме этого, сторона должна прекратить отправку и прием данных и закрыть соединение.

Оповещения описываются типом **Alert**, который определяется следующим образом:

```
struct {
    AlertLevel level;
    AlertDescription description;
} Alert;
```

Поля **Alert** имеют следующее значение:

- **level** — уровень оповещения: предупредительное или критическое;
- **description** — описание оповещения.

Типы вложенных в **Alert** полей определяются следующим образом:

```
enum { warning(1), fatal(2), (255) } AlertLevel;
```

```
enum {
    close_notify(0),
    unexpected_message(10),
    bad_record_mac(20),
    record_overflow(22),
    handshake_failure(40),
    bad_certificate(42),
    unsupported_certificate(43),
    certificate_revoked(44),
    certificate_expired(45),
    certificate_unknown(46),
    illegal_parameter(47),
    unknown_ca(48),
    access_denied(49),
    decode_error(50),
    decrypt_error(51),
    protocol_version(70),
    insufficient_security(71),
    internal_error(80),
    inappropriate_fallback(86),
    user_canceled(90),
    missing_extension(109),
    unsupported_extension(110),
    unrecognized_name(112),
    bad_certificate_status_response(113),
    unknown_psk_identity(115),
    certificate_required(116),
    no_application_protocol(120),
    (255)
} AlertDescription;
```

В перечислении `AlertLevel` значение `warning` соответствует предупредительному оповещению, значение `fatal` — критическому. Элементы `AlertDescription` описываются в 8.2 и 8.3.

8.2 Оповещения о закрытии соединения

Клиент и сервер должны информировать друг друга о закрытии соединения, чтобы избежать ситуации, в которой одна сторона считает, что соединение закрыто, а вторая сторона считает наоборот.

Определены следующие оповещения о закрытии соединения:

- `close_notify` — отправитель информирует получателя о том, что больше не будет посылать сообщения в данном соединении. Предполагается, что оповещение будет доставлено прежде, чем соединение будет закрыто на транспортном уровне;
- `user_canceled` — отправитель информирует получателя о том, что он отказывается от продолжения Handshake по причине, не связанной с ошибкой. Оповещение `user_canceled` следует использовать только в Handshake. Для информирования об отказе по завершении Handshake следует использовать `close_notify`. После `user_canceled` следует отправлять `close_notify`.

Указанные оповещения являются предупредительными.

Закрытие соединения может инициировать любая сторона. При закрытии она должна отправить оповещение `close_notify`, только если не отправляла ранее критическое оповещение. Любые данные, полученные после `close_notify`, должны игнорироваться.

Если соединение разорвано на транспортном уровне до приема `close_notify`, то получатель не может быть уверен, что он получил все предназначенные ему данные. Тем не менее получатель не обязан дожидаться `close_notify` перед закрытием соединения со своей стороны.

Если TLS используется в прикладном протоколе и по завершении TLS предполагается продолжение передачи данных на транспортном уровне, то сторона TLS должна дожидаться `close_notify` перед тем, как информировать прикладной протокол о завершении. Если продолжение не предполагается, то сторона может действовать по своему усмотрению.

8.3 Оповещения об ошибках

Если сторона TLS сталкивается с ошибкой, то ей следует отправить противоположной стороне соответствующее оповещение. После этого сторона должна закрыть соединение, прекратив отправку и прием данных. В настоящем стандарте фразы «закрыть соединение с оповещением X» и «прервать Handshake с оповещением X» означают, что если сторона отправляет оповещение, то это должно быть оповещение X.

Определены следующие оповещения об ошибках:

- `unexpected_message` — получено некорректное сообщение. Оповещение никогда не появится при штатном выполнении TLS;
- `bad_record_mac` — ошибка при снятии защиты с записи. Оповещение никогда не появится при штатном выполнении TLS, если только запись не была повреждена во время передачи;
- `record_overflow` — получена либо защищенная запись `TLSCiphertext`, размер которой больше $2^{14} + 256$ байт, либо открытая запись `TLSPlaintext`, размер которой больше 2^{14} байт. Максимальные размеры могут уменьшаться по предварительному согласованию сторон, например, с помощью расширения `max_fragment_length`. Оповещение никогда не появится при штатном выполнении TLS, если только запись не была повреждена во время передачи;
- `handshake_failure` — ни один из предложенных наборов параметров защиты не устраивает отправителя оповещения;
- `bad_certificate` — сертификат поврежден, содержит некорректную ЭЦП и т. д.;
- `unsupported_certificate` — сертификат данного типа не поддерживается;
- `certificate_revoked` — сертификат отозван выпустившей его стороной;
- `certificate_expired` — срок действия сертификата истек или еще не наступил;
- `certificate_unknown` — сертификат нельзя использовать из-за ошибки, которой нет среди перечисленных выше;
- `illegal_parameter` — в сообщении Handshake одно из полей лежит вне допустимого диапазона или не согласуется с другими полями. Оповещение касается сообщений, которые имеют корректный формат, но некорректное содержание;
- `unknown_ca` — получена корректная цепочка сертификатов или ее часть, но сертификат не был признан действительным, поскольку сертификат удостоверяющего центра не был найден или не соответствует известным точкам доверия;
- `access_denied` — получен действительный сертификат или PSK, но после проверки прав доступа отправитель оповещения решил не продолжать;

- **decode_error** — сообщение не может быть декодировано, так как одно из полей лежит вне допустимого диапазона или длина сообщения некорректна. Оповещение касается сообщений некорректного формата. Оповещение никогда не появится при штатном выполнении TLS, если только сообщение не было повреждено во время передачи;
- **decrypt_error** — неудачное завершение криптографической операции во время выполнения Handshake, в том числе отрицательный вердикт при проверке ЭЦП, сообщения Finished или PSK-скрепки;
- **protocol_version** — версия протокола, предложенная противоположной стороной при согласовании параметров защиты, не поддерживается;
- **insufficient_security** — используется вместо **handshake_failure** в тех случаях, когда процесс согласования параметров защиты завершился неудачно из-за того, что сервер запросил более высокий уровень защиты, чем тот, который поддерживает клиент;
- **internal_error** — внутренняя ошибка, которая не связана с противоположной стороной или логикой протокола (например, ошибка выделения памяти) и которая делает невозможным продолжение протокола;
- **inappropriate_fallback** — высылается сервером в рамках механизма TLS_FALLBACK_SCSV (см. [13]) в ответ на некорректную повторную попытку подключения со стороны клиента;
- **missing_extension** — согласованная версия TLS или другие параметры защиты предполагают включение в сообщение Handshake определенного расширения, но это расширение отсутствует;
- **unsupported_extension** — сообщение ServerHello или Finished содержит расширение, которое не было включено в предшествующее сообщение ClientHello или CertificateRequest;
- **unrecognized_name** — сервер, указанный клиентом в расширении **server_name**, не найден;
- **bad_certificate_status_response** — клиент сообщает серверу, что в расширении **status_request** содержится недействительный или неприемлемый ответ OCSP;
- **unknown_psk_identity** — сервер сообщает клиенту, что он не может определить предварительно согласованный секрет по идентификатору. Для информирования клиента серверу не обязательно использовать оповещение **unknown_psk_identity**. Вместо него сервер может использовать **decrypt_error**;
- **certificate_required** — сервер сообщает клиенту, что тот должен был представить сертификат, но не сделал этого;
- **no_application_protocol** — ни один из прикладных протоколов, указанных клиентом в расширении **application_layer_protocol_negotiation**, не поддерживается сервером.

Все перечисленные оповещения, а также любое неизвестное оповещение являются критическими. В реализациях TLS следует предусматривать ведение журнала отправляемых и получаемых оповещений.

9 Криптографическая поддержка

9.1 Алгоритмы Hash и HMAC

Стороны TLS используют алгоритм хэширования **Hash**, установленный в криптонаборе. Входными данными **Hash** является сообщение $X \in \{0,1\}^*$, выходными — хэш-значение $Y \in \{0,1\}^n$.

Должны соблюдаться следующие условия:

- длина n кратна 8 и $n/8 \leq 255$;
- сообщение X обрабатывается блоками по b бит;
- длина b кратна 8 и $b \geq n$.

Используется алгоритм имитозащиты **HMAC**. Алгоритм построен по одноименной схеме, установленной в СТБ 34.101.47. В **HMAC** в качестве базового алгоритма хэширования используется **Hash**.

Входными данными **HMAC** являются ключ $K \in \{0, 1\}^*$ и сообщение $X \in \{0, 1\}^*$, выходными — имитовставка $Y \in \{0, 1\}^n$. Размерности n и b базового алгоритма **Hash** определяют размерности выхода и внутренних структур данных **HMAC**.

Алгоритмы **Hash** и **HMAC**, а также длина их выходов n используются в других алгоритмах настоящего раздела. Число $n/8$, длина выходов в байтах, обозначается через **hash_length**.

9.2 Стенограммы

При криптографических вычислениях используются стенограммы **Handshake**. Стенограммы составляются из сообщений **Handshake**, переданных между сторонами к определенному моменту. Другими словами, стенограмма — это подпоследовательность следующей последовательности:

ClientHello, HelloRetryRequest, ClientHello₂, ServerHello, EncryptedExtensions, CertificateRequest, Certificate_S, CertificateVerify_S, Finished_S, EndOfEarlyData, Certificate_C, CertificateVerify_C, Finished_C.

Здесь нижний индекс 2 означает повторное сообщение, индекс S — сообщение сервера, C — клиента. В самом начале **Handshake** стенограмма представляет собой пустую последовательность $\perp$. Первое сообщение стенограммы — это обязательно **ClientHello**.

В сообщениях стенограммы сохраняются поля типа и длины (см. 6.1). Заголовки записей **Record**, в которые сообщения **Handshake** инкапсулируются, не учитываются. Сообщения интерпретируются как слова из $\{0, 1\}^{8*}$.

Стенограммы хэшируются с помощью алгоритма **Transcript-Hash**. Алгоритм получает на вход стенограмму $M = (M_1, M_2, \dots, M_m)$ и возвращает хэш-значение $Y \in \{0, 1\}^n$.

Шаги алгоритма:

- 1 Если M содержит сообщение **HelloRetryRequest**, то
 - 1) $M_1 \leftarrow \langle \text{message_type} \rangle_8 \parallel 0^{16} \parallel \langle \text{hash_length} \rangle_8 \parallel \text{Hash}(M_1)$, где **message_type** — константа, заданная в перечислении **HandshakeType** (см. 6.1).
- 2 Установить $Y \leftarrow \text{Hash}(M_1 \parallel M_2 \parallel \dots \parallel M_m)$.
- 3 Возвратить Y .

Примечание — На шаге 1.1 алгоритма $M_1 = \text{ ClientHello}$, $M_2 = \text{ HelloRetryRequest}$. Сообщение **ClientHello** заменяется искусственным сообщением типа **message_type**, которое содержит **Hash(ClientHello)**. Замена позволяет серверу без состояния перезапускать **Handshake**, не сохраняя **ClientHello**, а пересылая его хэш-значение в расширении **cookie** сообщения **HelloRetryRequest** (см. 6.3.3).

9.3 Алгоритмы построения ключей

9.3.1 Назначение

Для построения ключей используются алгоритмы **HKDF-Extract** и **HKDF-Expand** схемы **HKDF** (см. 5.2). С помощью **HKDF-Extract** по энтропийному слову X произвольной длины (ключевому материалу) и синхропосылке строится ключ K фиксированной длины. Затем

с помощью HKDF-Expand по ключу K и еще одной синхропосылке строится набор подчиненных ключей. Алгоритмы HKDF-Extract и HKDF-Expand можно вызывать рекурсивно, используя построенные ключи в качестве ключевого материала и синхропосылок при новых вызовах. В результате будет получен набор ключей, иерархически связанных друг с другом. Связь такова, что по подчиненному ключу вычислительно трудно определить как ключ, по которому он построен, так и другие подчиненные ключи.

Для учета специфики TLS алгоритм HKDF-Expand расширяется сначала до алгоритма HKDF-Expand-Label, а затем до Derive-Secret. При расширении усложняется интерфейс HKDF-Expand: вместо единой синхропосылки на вход подаются ее части. В первой части указывается метка (название) ключа, который строится, во второй — контекст, в котором построение выполняется. В Derive-Secret в качестве контекста выступает стенограмма протокола Handshake. Контекст может быть пустым.

9.3.2 Интерфейс

Входными данными HKDF-Extract являются ключевой материал $X \in \{0, 1\}^{8*}$ и синхропосылка $S \in \{0, 1\}^{8*}$. Выходными данными является ключ $K \in \{0, 1\}^n$.

Входными данными HKDF-Expand являются ключ $K \in \{0, 1\}^{8*}$, синхропосылка $S \in \{0, 1\}^{8*}$ и неотрицательное целое m — длина выхода в байтах. Выходными данными является подчиненный ключ $Y \in \{0, 1\}^{8m}$.

Входными данными HKDF-Expand-Label являются ключ $K \in \{0, 1\}^{8*}$, метка подчиненного ключа $L \in \{0, 1\}^{8*}$, контекст $C \in \{0, 1\}^{8*}$ и длина m . Выходными данными является подчиненный ключ $Y \in \{0, 1\}^{8m}$.

Входными данными Derive-Secret являются ключ $K \in \{0, 1\}^{8*}$, метка подчиненного ключа $L \in \{0, 1\}^{8*}$ и стенограмма M , которая представляет собой последовательность сообщений Handshake. Выходными данными является подчиненный ключ $Y \in \{0, 1\}^n$.

Должны соблюдаться следующие ограничения на входные данные:

- длина m не должна превосходить $255n/8$;
- длина метки L в байтах должна лежать в диапазоне от 1 до 249;
- длина контекста C в байтах не должна превосходить 255.

Метки ключей кодируются строками ASCII с окаймлением двойными кавычками (см. раздел 4). В алгоритме HKDF-Expand-Label к меткам добавляется префикс "tls13".

9.3.3 Алгоритм HKDF-Extract

Вычисление HKDF-Extract(X, S) выполняется следующим образом:

- 1 Если $S = \perp$, то $S \leftarrow 0^n$.
- 2 Установить $K \leftarrow \text{HMAC}(S, X)$.
- 3 Возвратить K .

9.3.4 Алгоритм HKDF-Expand

Вычисление HKDF-Expand(K, S, m) выполняется следующим образом:

- 1 Установить $(Y, T) \leftarrow (\perp, \perp)$ (пустые строки).
- 2 Для $i = 1, 2, \dots, \lceil 8m/n \rceil$:
 - 1) $T \leftarrow \text{HMAC}(K, T \parallel S \parallel \langle i \rangle_8)$;
 - 2) $Y \leftarrow Y \parallel T$.
- 3 Установить $Y \leftarrow \text{Lo}(Y, 8m)$.
- 4 Возвратить Y .

9.3.5 Алгоритм HKDF-Expand-Label

В алгоритме HKDF-Expand-Label используется структура данных HkdfLabel:

```
struct {
    uint16 length;
    opaque label<7..255>;
    opaque context<0..255>;
} HkdfLabel;
```

Вычисление HKDF-Expand-Label(K, L, C, m) выполняется следующим образом:

- 1 Построить структуру HkdfLabel, заполнив ее поля следующим образом:
 - 1) HkdfLabel.length $\leftarrow m$;
 - 2) HkdfLabel.label $\leftarrow$ "tls13 " $\parallel L$;
 - 3) HkdfLabel.context $\leftarrow C$.
- 2 Установить $Y \leftarrow$ HKDF-Expand($K, \text{HkdfLabel}, m$).
- 3 Возвратить Y .

9.3.6 Алгоритм Derive-Secret

Вычисление Derive-Secret(K, L, M) выполняется следующим образом:

- 1 Установить $H \leftarrow$ Transcript-Hash(M).
- 2 Установить $Y \leftarrow$ HKDF-Expand-Label($K, L, H, \text{hash_length}$).
- 3 Возвратить Y .

9.4 Расписание ключей

В протоколе Handshake стороны определяют один или несколько общих секретов, по которым затем строят подчиненные общие ключи. К общим секретам относятся предварительно согласованный секрет PSK и ключ DHE, выработанный по протоколу Диффи — Хеллмана. Перечень общих секретов определяется выбранным режимом: PSK, DHE или PSK+DHE (см. 5.2). Если тот или иной общий секрет не формируется, то он полагается равным пустому слову.

При построении подчиненных ключей используются стенограммы Handshake. Поскольку приветственные сообщения Handshake содержат случайные данные, подчиненные ключи будут волатильны, даже если общие секреты повторяются, например когда в нескольких соединениях используется один и тот же секрет PSK, не сопровождаемый ключом DHE.

Подчиненные ключи строятся в соответствии с расписанием, определенным на рисунке 5. Здесь Finished_S — сообщение Finished сервера, Finished_C — клиента.

Ключи $K_1, \dots, K_5$ являются вспомогательными, они не используются за пределами расписания. По K_1 (ранний секрет), K_3 (секрет Handshake) и K_5 (главный секрет) строятся ключи, представленные в таблице 2. При построении явно учитываются стенограммы Handshake. Ключи таблицы используются для построения подчиненных ключей и других объектов за пределами расписания. При этом стенограммы продолжают учитываться, но только опосредованно.

При построении binder_key должна использоваться строка "ext binder", если используется внешний PSK, согласованный за пределами TLS, и строка "res binder", если PSK — это секрет для возобновления связи, построенный по resumption_master_secret в одном из предыдущих соединений.

Клиент вычисляет ключ K_1 для каждого PSK, который он намеревается использовать. Это необходимо потому, что по K_1 нужно построить сначала ключ binder_key, а затем

$$\begin{aligned}
K_1 &\leftarrow \text{HKDF-Extract}(\text{PSK}, \perp), \\
\text{binder_key} &\leftarrow \text{Derive-Secret}(K_1, \text{"ext binder"/"res binder"}, \\
&\quad \perp), \\
\text{client_early_traffic_secret} &\leftarrow \text{Derive-Secret}(K_1, \text{"c e traffic"}, \\
&\quad (\text{ClientHello})), \\
\text{early_exporter_master_secret} &\leftarrow \text{Derive-Secret}(K_1, \text{"e exp master"}, \\
&\quad (\text{ClientHello})), \\
K_2 &\leftarrow \text{Derive-Secret}(K_1, \text{"derived"}, \perp), \\
K_3 &\leftarrow \text{HKDF-Extract}(\text{DHE}, K_2), \\
\text{client_handshake_traffic_secret} &\leftarrow \text{Derive-Secret}(K_3, \text{"c hs traffic"}, \\
&\quad (\text{ClientHello}, \dots, \text{ServerHello})), \\
\text{server_handshake_traffic_secret} &\leftarrow \text{Derive-Secret}(K_3, \text{"s hs traffic"}, \\
&\quad (\text{ClientHello}, \dots, \text{ServerHello})), \\
K_4 &\leftarrow \text{Derive-Secret}(K_3, \text{"derived"}, \perp), \\
K_5 &\leftarrow \text{HKDF-Extract}(\perp, K_4). \\
\text{client_application_traffic_secret} &\leftarrow \text{Derive-Secret}(K_5, \text{"c ap traffic"}, \\
&\quad (\text{ClientHello}, \dots, \text{Finished}_S)), \\
\text{server_application_traffic_secret} &\leftarrow \text{Derive-Secret}(K_5, \text{"s ap traffic"}, \\
&\quad (\text{ClientHello}, \dots, \text{Finished}_S)), \\
\text{exporter_master_secret} &\leftarrow \text{Derive-Secret}(K_5, \text{"exp master"}, \\
&\quad (\text{ClientHello}, \dots, \text{Finished}_S)), \\
\text{resumption_master_secret} &\leftarrow \text{Derive-Secret}(K_5, \text{"res master"}, \\
&\quad (\text{ClientHello}, \dots, \text{Finished}_C)).
\end{aligned}$$

Рисунок 5 — Расписание ключей

PSK-скрепку для включения в `ClientHello`. После ответа сервера клиент оставляет только один ключ K_1 — тот, который соответствует PSK, выбранному сервером. Если сервер отказался от использования режима PSK, то клиенту придется еще раз вычислить K_1 , на этот раз для пустого PSK.

Ключ расписания, по которому уже построены все подчиненные ключи, следует уничтожить.

9.5 Обновление ключей

По завершении Handshake стороны формируют общие ключи `client_application_traffic_secret` и `server_application_traffic_secret`, а затем строят по ним ключи защиты прикладных данных.

Впоследствии клиент может обновить `client_application_traffic_secret` и построить по нему новый ключ защиты данных, отправляемых серверу. Соответственно, сервер может обновить `server_application_traffic_secret` и построить новый ключ защиты данных, отправляемых клиенту.

Таблица 2 — Назначение ключей

Ключ	Подчиненные объекты (строятся по ключу)
<code>binder_key</code>	PSK-скрепка (см. 9.8)
<code>client_early_traffic_secret</code>	Ключ и начальная синхропосылка ранней защиты (см. 9.6)
<code>early_exporter_master_secret</code>	Ранний экспортер (см. 9.9.1)
<code>client_handshake_traffic_secret</code>	Ключ и начальная синхропосылка защиты сообщений Handshake, отправляемых клиентом (см. 9.6)
<code>server_handshake_traffic_secret</code>	Ключ и начальная синхропосылка защиты сообщений Handshake, отправляемых сервером (см. 9.6)
<code>client_application_traffic_secret</code>	Ключ и начальная синхропосылка защиты прикладных данных, отправляемых клиентом (см. 9.6)
<code>server_application_traffic_secret</code>	Ключ и начальная синхропосылка защиты прикладных данных, отправляемых сервером (см. 9.6)
<code>exporter_master_secret</code>	Экспортер (см. 9.9.1)
<code>resumption_master_secret</code>	PSK для возобновления связи (см. 9.9.2)

Правила обновления:

```

client_application_traffic_secret ← HKDF-Expand-Label(
    client_application_traffic_secret,
    "traffic upd", ⊥, hash_length),
server_application_traffic_secret ← HKDF-Expand-Label(
    server_application_traffic_secret,
    "traffic upd", ⊥, hash_length).

```

Перед обновлением ключей сторона-инициатор должна отправить противоположной стороне сообщение `KeyUpdate` (см. 6.7.3).

После обновления следует удалить предыдущие ключи.

9.6 Ключи защиты записей

Записи протокола `Record` защищаются с помощью алгоритма аутентифицированного шифрования, заданного в действующем криптонаборе. Для защиты записей, отправляемых от клиента серверу, используются ключ `client_write_key` и начальная синхропосылка `client_write_iv`. Ключ используется напрямую, а `client_write_iv` складывается с порядковым номером записи для получения актуальной синхропосылки (см. 7.4).

Пары (`client_write_key`, `client_write_iv`) строятся по каждому из ключей таблицы 2, имя которого начинается с `client`. Назначение исходного ключа определяет, для защиты записей какого типа должна использоваться построенная по нему пара.

Построение выполняется следующим образом:

```

client_write_key ← HKDF-Expand-Label( $K$ , "key", ⊥, key_length),
client_write_iv ← HKDF-Expand-Label( $K$ , "iv", ⊥, iv_length).

```

Здесь K — исходный ключ (один из ключей `client_early_traffic_secret`, `client_handshake_traffic_secret` или `client_application_traffic_secret`), `key_length` — длина в байтах ключа алгоритмов аутентифицированного шифрования, `iv_length` — длина в байтах синхропосылки.

Ключ и начальная синхропосылка пересчитываются всякий раз, когда исходный ключ K обновляется (см. 9.5).

Введенные правила распространяются на ключи и начальные синхропосылки, которые используются для защиты записей, отправляемых от сервера клиенту. Изменяются только имена объектов: префикс `server` вместо префикса `client`. В частности,

$$\begin{aligned}\text{server_write_key} &\leftarrow \text{HKDF-Expand-Label}(K, \text{"key"}, \perp, \text{key_length}), \\ \text{server_write_iv} &\leftarrow \text{HKDF-Expand-Label}(K, \text{"iv"}, \perp, \text{iv_length}),\end{aligned}$$

где K — один из ключей `server_handshake_traffic_secret` или `server_application_traffic_secret`.

9.7 Аутентификаторы

Для подтверждения своей подлинности и (или) демонстрации владения общими ключами стороны вычисляют аутентификаторы — подписи и имитовставки стенограмм Handshake. Подпись размещается в поле `signature` сообщения `CertificateVerify`, имитовставка — в поле `verify_data` сообщения `Finished`. Клиент может отправлять сообщения `CertificateVerify` и `Finished` как в процессе выполнения Handshake, так и после (см. 6.7.2).

Подписывается результат конкатенации следующих объектов:

- 1) строка из 64 октетов 20_{16} (пробел в таблице ASCII);
- 2) строка `"TLS 1.3, server CertificateVerify"`, если подписантом является сервер, или строка `"TLS 1.3, client CertificateVerify"`, если подписантом является клиент;
- 3) октет 00_{16} (разделитель);
- 4) хэш-значение `Transcript-Hash(ClientHello, ..., Certificate)`, где `Certificate` — сообщение подписанта, непосредственно предшествующее его же сообщению `CertificateVerify`, в котором будет размещаться подпись.

Примечание — Выбранный формат подписываемых данных защищает от угроз для сервера, который поддерживает соединения TLS версии 1.2, используя тот же сертификат и, соответственно, личный ключ. При выборе в TLS версии 1.2 режима DHE сервер подписывает и отправляет в сообщении `ServerKeyExchange` данные, начальные 32 байта которых контролирует клиент.

Пример — Если хэш-значение состоит из 32 октетов 01_{16} , то сервер будет подписывать следующее двоичное слово:

```
20202020202020202020202020202020202020202020202020202020202020202020
20202020202020202020202020202020202020202020202020202020202020202020
544c5320312e332c207365727665722043657274696669636177465566572696679
00
0101010101010101010101010101010101010101010101010101010101010101010116.
```

Стенограмма, для которой вычисляется имитовставка, завершается сообщением, указанным в таблице 3. В таблице нижний индекс S означает сообщения сервера, индекс C — клиента. В определенных случаях имеется несколько вариантов завершающего сообщения. Следует выбирать сообщение, которое отправляется в Handshake позднее других.

Таблица 3 — Имитовставки стенограмм

Сторона	Окончание стенограммы	Ключ <code>base_key</code>
Сервер	EncryptedExtensions CertificateRequest	<code>server_handshake_traffic_secret</code>
Клиент (Handshake)	Finished _s EndOfEarlyData CertificateVerify _C	<code>client_handshake_traffic_secret</code>
Клиент (после Handshake)	CertificateVerify _C	<code>client_application_traffic_secret</code>

При вычислении имитовставки стенограммы используется ключ `base_key`, также представленный в таблице 3. Вычисления выполняются следующим образом:

$$\begin{aligned}
 H &\leftarrow \text{Transcript-Hash}(M), \\
 \text{finished_key} &\leftarrow \text{HKDF-Expand-Label}(\text{base_key}, \text{"finished"}, \perp, \text{hash_length}), \\
 \text{verify_data} &\leftarrow \text{HMAC}(\text{finished_key}, H).
 \end{aligned}$$

Здесь M — исходная стенограмма, `verify_data` — искомая имитовставка.

9.8 PSK-скрепки

PSK-скрепка устанавливает связь между секретом PSK и текущим сеансом Handshake. Если секрет был согласован в предыдущем соединении (с помощью `NewSessionTicket`), то дополнительно устанавливается связь с этим соединением.

PSK-скрепки вычисляются при подготовке расширения `pre_shared_key` в момент отправки первого или повторного сообщения `ClientHello`. Расширение описывается типом `OfferedPsk`, определенным в 6.3.13, и состоит из полей `identities` и `binders`. Скрепки сохраняются в последнем поле. Расширение `pre_shared_key` является последним в списке расширений `ClientHello` и, следовательно, завершает сообщение.

При вычислении PSK-скрепки используется текущая стенограмма Handshake. Она заканчивается текущим сообщением `ClientHello`: первым или повторным. Стенограмма сокращается: в расширении `pre_shared_key` завершающего `ClientHello` исключается поле `binders`.

Сокращенная стенограмма хэшируется. В результате получается хэш-значение

$$H \leftarrow \text{Transcript-Hash}(\text{Truncate}(\text{ClientHello})),$$

если Handshake выполняется без перезапуска, и хэш-значение

$$H \leftarrow \text{Transcript-Hash}(\text{ClientHello}, \text{HelloRetryRequest}, \text{Truncate}(\text{ClientHello}_2)),$$

если с перезапуском. Здесь `Truncate` — функция исключения `binders`, `ClientHello2` — повторное приветственное сообщение клиента.

После определения H скрепка `binder` вычисляется следующим образом:

$$\begin{aligned}
 \text{finished_key} &\leftarrow \text{HKDF-Expand-Label}(\text{binder_key}, \text{"finished"}, \perp, \text{hash_length}), \\
 \text{binder} &\leftarrow \text{HMAC}(\text{finished_key}, H).
 \end{aligned}$$

9.9 Другие ключи и секреты

9.9.1 Экспортеры

По завершении TLS стороны могут продолжить использовать общий ключевой материал с помощью экспортеров. Экспортер — это ключ, который строится по специальному общему ключу, согласованному сторонами в Handshake. Экспортер характеризуется меткой и строится с учетом определенного контекста. Изменяя метку или контекст, можно получить семейство экспортеров. При раскрытии одного из представителей семейства сохраняется секретность как исходного ключа, так и других представителей.

Алгоритм построения экспортера принимает на вход ключ K , метку $L \in \{0,1\}^{8*}$, контекст $C \in \{0,1\}^{8*}$ и натуральное число m — длину экспортера в байтах. Алгоритм возвращает экспортер $Y \in \{0,1\}^{8m}$. В качестве K должен использоваться один из ключей `early_exporter_master_secret` или `exporter_master_secret` (см. таблицу 2).

Шаги алгоритма:

- 1 Установить $H \leftarrow \text{Hash}(C)$.
- 2 Установить $R \leftarrow \text{Derive-Secret}(K, L, \perp)$.
- 3 Установить $Y \leftarrow \text{HKDF-Expand-Label}(R, \text{"exporter"}, H, m)$.
- 4 Возвратить Y .

При вызовах `Derive-Secret` и `HKDF-Expand-Label` должны соблюдаться ограничения этих алгоритмов (см. 9.3.2). В частности длина метки L в байтах не должна превышать 249. Дополнительные ограничения на метки экспортеров установлены в [14].

При выборе исходного ключа K стороны должны отдавать предпочтение `exporter_master_secret`, если иное не оговорено явно. Ключ `early_exporter_master_secret` введен для того, чтобы иметь возможность строить экспортеры на ранней стадии (в механизме 0-RTT), когда `exporter_master_secret` еще не сформирован. В реализациях TLS рекомендуется выносить построение ранних экспортеров в отдельный сервис.

9.9.2 Секреты для возобновления связи

Секрет для возобновления связи строится по ключу `resumption_master_secret` и синхропосылке `ticket_nonce`, включенной в сообщение `NewSessionTicket` (см. 6.7.1).

Секретом является следующее значение:

$$\text{HKDF-Expand-Label}(\text{resumption_master_secret}, \text{"resumption"}, \\ \text{ticket_nonce}, \text{hash_length}).$$

Приложение А (обязательное) Правила описания данных

А.1 Общие соглашения

Комментарий начинается парой символов `/*` и заканчивается парой символов `*/`.

Необязательные компоненты окаймляются двойными квадратными скобками: `[[ ]]`.

Данные описываются типами. Имеются базовые (примитивные) типы, от которых порождаются составные.

Данные кодируются последовательностями байтов. Правила кодирования определяются типом данных. Байты кодового представления записываются и читаются слева направо и сверху вниз.

Базовый тип `opaque` описывает один байт неструктурированных данных.

Псевдоним `T1` уже введенного типа `T` определяется следующим образом:

```
T T1;
```

А.2 Числа

Целые числа от 0 до 255, беззнаковые байты, описываются типом `uint8`.

Порожденные от `uint8` типы (векторы элементов `uint8`, см. А.3):

```
uint8 uint16 [2];
```

```
uint8 uint24 [3];
```

```
uint8 uint32 [4];
```

```
uint8 uint64 [8];
```

Они также описывают беззнаковые числа, кодируя их последовательностями байтов фиксированной длины. Используются соглашения «от старших к младшим» (big-endian): первый байт считается старшим байтом числа, последний — младшим.

Числа записываются либо в десятичном, либо в шестнадцатеричном виде. Признаком шестнадцатеричной записи является префикс `0x`. Например, `0x01020304` — это шестнадцатеричная запись числа $16909060 = 1 \cdot 2^{24} + 2 \cdot 2^{16} + 3 \cdot 2^8 + 4$. При этом число кодируется байтами `0x01`, `0x02`, `0x03`, `0x04`.

Число `a` в степени `b` обозначается через `ab`.

А.3 Векторы

Вектор — это одномерный массив однотипных элементов. Длина вектора определяется либо заранее, либо в момент кодирования. В любом случае длина определяет размер вектора в байтах, а не число элементов в векторе.

Тип `T2`, который описывает векторы фиксированной длины с элементами типа `T`, определяется следующим образом:

```
T T2 [n];
```

Здесь `n` — размер векторов `T2` в байтах. Предполагается, что коды элементов `T` имеют фиксированную длину и что число `n` кратно этой длине.

При кодировании векторов фиксированной длины длина не включается в кодовое представление.

В следующем примере тип `Datum` описывает тройку неструктурированных байтов, а тип `Data` — тройку элементов типа `Datum`, которые в сумме занимают 9 байт:

```
opaque Datum[3];
Datum Data[9];
```

Векторы переменной длины описываются путем задания диапазона допустимых длин (включая границы) с помощью обозначения `<floor..ceiling>`:

```
T T3<floor..ceiling>;
```

Число `floor` задает минимальную длину, `ceiling` — максимальную.

В кодовом представлении вектора переменной длины указывается сначала фактическая длина вектора, а затем его содержимое. Код длины занимает столько же байтов, сколько требуется для кодирования числа `ceiling`. Вектор переменной длины, актуальная длина которого равняется нулю, кодируется пустой последовательностью.

В следующем примере вектор типа `mandatory` содержит от 300 до 400 байт, а вектор типа `longer` — до 800 байт или до 400 элементов типа `uint16`:

```
opaque mandatory<300..400>;
uint16 longer<0..800>;
```

Вектор `mandatory` не может быть пустым. Код его длины занимает 2 байта, которых достаточно для представления числа 400 (см. А.2). Вектор `longer` может иметь нулевую длину, и тогда его кодовое представление — это пустая последовательность. Если же длина ненулевая, то кодовое представление вектора начинается с кода длины и этот код снова является двухбайтовым.

А.4 Перечисления

Тип, определяемый с помощью ключевого слова `enum`, описывает перечисление. Переменные типа принимают неотрицательные целые значения, причем только те, которые объявлены в определении типа. Этим значениям назначаются имена. Сравнение и присваивание должны выполняться в рамках одного типа.

В следующем примере тип `Te` задает использование значений `v1`, `v2`, ..., `vn` с именами `e1`, `e2`, ..., `en`:

```
enum { e1(v1), e2(v2), ..., en(vn) [[, (m)]] } Te;
```

Перечень допустимых значений и соответствующих имен перечислительного типа может дополняться. Дополнение задается оператором `+=`:

```
Te += {ee(vv), eee(vvv)};
```

Примечание — Использование оператора `+=` является расширением правил описания данных, заданных в [3].

В определении перечислительного типа может фигурировать максимальное допустимое значение. Оно указывается без имени и используется при определении длины кодового представления элементов типа. В примере с `Te` таким значением является `m`.

Код переменной перечислительного типа занимает столько байтов, сколько необходимо для записи максимального допустимого значения. Элементы следующего перечисления занимают 1 байт:

```
enum { red(3), blue(5), white(7) } Color;
```

Максимальное допустимое значение может быть указано в определении типа без введения соответствующего элемента, т. е. без имени. В примере с типом `Te` таким значением является `m`.

В следующем примере коды переменных типа `Taste` будут занимать 2 байта, хотя могут принимать только значения 1, 2 или 4:

```
enum { sweet(1), sour(2), bitter(4), (32000) } Taste;
```

Областью видимости имен элементов перечисления является сам перечислительный тип. При ссылке на имя можно указывать этот тип явно или опускать его, если он ясен из контекста:

```
Color color = Color.blue;      /* задано явно */
Color color = blue;            /* определяется неявно */
```

Имена, назначаемые элементам перечислительного типа, не обязательно должны быть уникальны, имя может быть назначено целому диапазону значений. Диапазон задается минимальной и максимальной границами, разделенными двумя точками:

```
enum { sad(0), meh(1..254), happy(255) } Mood;
```

A.5 Структуры

Примитивные типы могут объединяться в составные типы — структуры. Синтаксис определения структур аналогичен синтаксису языка Си:

```
struct {
    T1 f1;
    T2 f2;
    ...
    Tn fn;
} T;
```

В структуры могут быть вложены другие структуры, векторы фиксированной и переменной длин. Структуры `V1` и `V2` в примерах раздела A.7 это демонстрируют.

При ссылке на поле структуры указываются тип и имя поля. Например, `T.f2` — ссылка на второе поле структуры, определенной выше.

Кодовое представление переменной составного типа представляет собой объединение кодов ее последовательных полей.

A.6 Константы

Полям структур и переменным могут назначаться значения. Оператор присваивания обозначается знаком `=`. Например,

```
struct {
    T1 f1 = 8; /* T.f1 всегда равняется 8 */
    T2 f2;
} T;
```

A.7 Варианты

Структура может иметь разные варианты построения. Выбор варианта производится на основании дополнительной информации, которая оговаривается при описании структуры за рамками данных правил.

Переменная, на основании которой производится выбор варианта, должна иметь перечислительный тип. Переключение между вариантами описывается с помощью ключевого слова `select`. По каждой ветке `select` указывается тип, соответствующий варианту построения, и, возможно, имя поля данного типа:

```

struct {
    T1 f1;
    T2 f2;
    ...
    Tn fn;
    select (E) {
        case e1: Te1 [[fe1]];
        case e2: Te2 [[fe2]];
        ...
        case en: Ten [[fen]];
    };
} Tv;

```

В следующем примере за выбор варианта в структуре `VariantRecord` отвечает переменная перечислительного типа `VariantTag`:

```

enum { apple(0), orange(1) } VariantTag;

struct {
    uint16 number;
    opaque string<0..10>;    /* переменная длина */
} V1;

struct {
    uint32 number;
    opaque string[10];        /* фиксированная длина */
} V2;

struct {
    VariantTag type;
    select (VariantRecord.type) {
        case apple: V1;
        case orange: V2;
    };
} VariantRecord;

```

Приложение Б (обязательное) Расширение `extended_master_secret`

Б.1 Назначение

В настоящем приложении определяется расширение `extended_master_secret`, введенное в [15]. Расширение защищает реализации TLS версии 1.2 и ниже от атаки «Triple Handshake» [16]. Использование расширения `extended_master_secret` является дополнительным обязательным требованием СТБ 34.101.65.

Протокол TLS версии 1.3 по построению защищен от атаки «Triple Handshake» и в применении расширения `extended_master_secret` нет необходимости. Тем не менее расширение может использоваться, чтобы информировать о защите прикладные протоколы.

Б.2 Вычисление мастер-ключа

Если клиент и сервер согласовали применение расширения `extended_master_secret`, то они должны использовать следующий алгоритм вычисления мастер-ключа:

1 Построить стенограмму `handshake_messages`, включив в нее сообщения Handshake, начиная с `ClientHello` и заканчивая `ClientKeyExchange`.

2 Определить алгоритм хэширования `Hash`, используемый в алгоритме генерации псевдослучайных чисел (см. СТБ 34.101.65 [пункт 8.16]), и вычислить с его помощью хэш-значение `session_hash = Hash(handshake_messages)` стенограммы `handshake_messages`.

3 Вычислить мастер-ключ

```
master_secret = PRF(pre_master_secret,  
    "extended master secret", session_hash)[0..47];
```

Если применение расширения `extended_master_secret` не было согласовано, то клиент и сервер должны использовать алгоритм вычисления мастер-ключа, определенный в СТБ 34.101.65 (пункт 8.16).

Б.3 Установка связи

Действия сторон при установке связи:

1 Клиент, поддерживающий расширение `extended_master_secret`, должен отправить его в сообщении `ClientHello`.

2 Сервер, получивший расширение `extended_master_secret` и поддерживающий его, должен включить расширение в сообщение `ServerHello`.

3 Если сервер получает `ClientHello` без расширения `extended_master_secret`, то ему следует прервать Handshake. Если сервер все-таки решает продолжить протокол, то он не должен включать расширение в `ServerHello`.

4 Если клиент получает `ServerHello` без расширения `extended_master_secret`, то ему следует прервать Handshake.

Б.4 Сокращенная установка связи

Действия сторон при сокращенной установке связи:

1 Если в предыдущем сеансе Handshake не использовалось расширение `extended_master_secret`, то клиенту не следует возобновлять такой сеанс. Вместо этого клиенту следует инициировать полную установку связи.

2 Если сервер получает `ClientHello` с расширением `extended_master_secret`, но расширение не использовалось в предыдущем сеансе, то сервер должен прервать Handshake.

3 Если сервер получает `ClientHello` без расширения `extended_master_secret`, но расширение использовалось в предыдущем сеансе, то сервер должен прервать Handshake.

4 Если сервер получает `ClientHello` без расширения `extended_master_secret` и расширение не использовалось в предыдущем сеансе, то серверу следует прервать Handshake.

5 Если сервер получает `ClientHello` с расширением `extended_master_secret` и расширение использовалось в предыдущем сеансе, то сервер должен включить расширение в `ServerHello`.

6 Если клиент получает `ServerHello` с расширением `extended_master_secret`, но расширение не использовалось в предыдущем сеансе, то клиент должен прервать Handshake.

7 Если клиент получает `ServerHello` без расширения `extended_master_secret`, но расширение использовалось в предыдущем сеансе, то клиент должен прервать Handshake.

Б.5 Идентификатор

Расширению `extended_master_secret` назначается идентификатор 23. Он указывается в поле `extension_type` структуры `Extension`. Соответствующее поле `extension_data` должно быть пустым.

Приложение В
(обязательное)
Использование алгоритмов стандартов серии СТБ 34.101

В.1 Назначение

В настоящем приложении определяются правила использования в TLS криптографических алгоритмов, установленных в стандартах серии СТБ 34.101. Правила касаются криптонаборов (см. раздел В.2), протокола Диффи — Хеллмана (см. раздел В.3), групп точек эллиптической кривой (см. раздел В.3), алгоритмов ЭЦП (см. раздел В.5).

Требования настоящего приложения являются обязательными только для тех реализаций TLS, в которых используются алгоритмы СТБ.

В.2 Криптонаборы

Должны использоваться криптонаборы из следующего списка (см. таблицу В.1):

1 `TLS_BELT_CHE256_BELT_HASH` — алгоритмы аутентифицированного шифрования `belt-che256` в связке с алгоритмом хэширования `belt-hash`. Алгоритмы определены в СТБ 34.101.31.

2 `TLS_BASH_PRG_AE2561_BASH256` — алгоритмы аутентифицированного шифрования `bash-prg-ae2561` в связке с алгоритмом хэширования `bash256`. Алгоритмы определены в СТБ 34.101.77.

Криптонабор `TLS_BASH_PRG_AE2561_BASH256` обязателен к реализации.

Таблица В.1 — Криптонаборы

Криптонабор	Идентификатор
<code>TLS_BELT_CHE256_BELT_HASH</code>	{255, 29}
<code>TLS_BASH_PRG_AE2561_BASH256</code>	{255, 30}

Алгоритмы криптонаборов имеют следующие размерности:

- `key_length` — длина в байтах ключа алгоритмов аутентифицированного шифрования, равняется 32;
- `iv_length` — длина в байтах синхропосылки алгоритмов аутентифицированного шифрования, равняется 16;
- `hash_length` — длина в байтах выхода алгоритма хэширования, равняется 32.

При аутентифицированном шифровании в протоколе Record ассоциированные данные составляются из полей `opaque_type`, `legacy_record_version` и `length` структуры `TLSCiphertext`. Размер ассоциированных данных — 5 байт.

В алгоритмах `bash-prg-ae2561` ассоциированные данные $I \in \{0, 1\}^{40}$ переносятся в анонс A , объединяясь с синхропосылкой $S \in \{0, 1\}^{128}$:

$$A \leftarrow S \parallel I \parallel 0^{24}.$$

Шаг 2.1 с вызовом $\alpha.\text{absorb}(I)$ исключается и при установке защиты, и при ее снятии.

Примечание — Анонс дополняется нулями, чтобы соответствовать требованиям `bash-prg-ae2561`: длина анонса в битах должна быть кратна 32.

В алгоритмах `belt-che256` ассоциированные данные и синхропосылка используются обычным образом, в соответствии с интерфейсом алгоритмов.

Имитовставки **belt-che256** состоят из 8 байт, имитовставки **bash-prg-ae2561** — из 32. При использовании алгоритмов в TLS имитовставки не должны сокращаться. Имитовставка T добавляется к шифртексту Y , образуя аутентифицированный шифртекст $Y \parallel T$, который размещается в `TLSCiphertext.encrypted_record`.

Ключ **belt-che256** должен обновляться при достижении порога в 2^{32} защищенных записей. Ключ **bash-prg-ae2561** используется безлимитно.

Примечание — Максимальное число записей **belt-che256** определено в соответствии с СТБ 34.101.31 (приложение В) на среднем уровне гарантий. На этом уровне квота ключа составляет $2^{48} \sqrt{2/(5D+7)}$ 16-байтовых блоков аутентифицированного шифртекста, где D — максимальное число блоков, обрабатываемых за один вызов **belt-che256**. В контексте Record величина $D = 1 + 1025 + 1 = 1027$: один неполный блок ассоциированных данных, не более $\lceil (2^{14} + 1)/16 \rceil = 1025$ блоков в `TLSCiphertext.encrypted_record` и один служебный блок **belt-che256**. При таком D квота ключа составляет $2^{42.33}$ блоков. Аутентифицированный шифртекст в `TLSCiphertext.encrypted_record` состоит из не более чем $1025 + 1 = 1026$ блоков: не более $\lceil (2^{14} + 1)/16 \rceil = 1025$ блоков шифртекста и один неполный блок с имитовставкой. Поэтому без смены ключа разрешается защитить как минимум $2^{42.33}/1026 > 2^{32}$ записей Record.

С секретом, предварительно согласованным за пределами TLS, должен быть ассоциирован алгоритм хэширования (см. 6.3.13). В качестве такого алгоритма должен использоваться либо **belt-hash**, либо **bash256**. По умолчанию, если ассоциированный алгоритм хэширования не задан явно, должен использоваться **belt-hash**.

В.3 Протокол Диффи — Хеллмана

Протокол Диффи — Хеллмана должен быть реализован в соответствии со следующими правилами, установленными в СТБ 34.101.66 и основанными на соглашениях и алгоритмах, установленных в СТБ 34.101.45.

1 В качестве базовой циклической группы должна использоваться группа точек эллиптической кривой, которая удовлетворяет соглашениям СТБ 34.101.45.

2 Одноразовые личный и открытый ключи должны генерироваться с помощью алгоритма **bign-genkeypair** (см. СТБ 34.101.45).

3 Стороны должны проверять открытые ключи друг друга с помощью алгоритма **bign-valpubkey** (см. СТБ 34.101.45).

4 Стороны должны вычислять общий ключ с помощью алгоритма **bake-dh** (СТБ 34.101.66): общий ключ $K = uV$, где u — одноразовый личный ключ (скаляр), V — одноразовый открытый ключ противоположной стороны (точка кривой).

5 Общий ключ K , аффинная точка эллиптической кривой, должен кодироваться строкой октетов по правилам, заданным в СТБ 34.101.45 (пункт 5.4). Кодировются обе координаты K , код x -координаты указывается первым. При кодировании координат используются правила «от младших к старшим» (little-endian), незначащие нули не отбрасываются. При использовании эллиптической кривой уровня стойкости ℓ кодовое представление K состоит из $\ell/2$ октетов.

В.4 Группы точек эллиптической кривой

В протоколе Диффи — Хеллмана должны использоваться следующие группы точек эллиптической кривой, которые удовлетворяют соглашениям СТБ 34.101.45 и определяются стандартными параметрами, установленными в СТБ 34.101.45 (приложение Б):

- **bign-curve256v1** — стандартные параметры уровня стойкости $\ell = 128$;
- **bign-curve384v1** — стандартные параметры уровня стойкости $\ell = 192$;
- **bign-curve512v1** — стандартные параметры уровня стойкости $\ell = 256$.

Идентификаторы групп добавляются в перечисление `NamedGroup` (см. 6.3.9):

```
NamedGroup += {
    bign_curve256v1(0xFE01),
    bign_curve384v1(0xFE02),
    bign_curve512v1(0xFE03)
};
```

Идентификаторы используются в расширениях `supported_groups` и `key_share`.

В.5 Алгоритмы электронной цифровой подписи

Должны использоваться следующие алгоритмы выработки и проверки ЭЦП, установленные в СТБ 34.101.45 и сопровождаемые алгоритмами хэширования СТБ 34.101.31 и СТБ 34.101.77:

- `bign-with-hbelt` — алгоритмы ЭЦП в связке с алгоритмом хэширования `belt-hash` (см. СТБ 34.101.31);
- `bign-with-bash384` — алгоритмы ЭЦП в связке с алгоритмом хэширования `bash384` (см. СТБ 34.101.77);
- `bign-with-bash512` — алгоритмы ЭЦП в связке с алгоритмом хэширования `bash512` (см. СТБ 34.101.77).

В алгоритмах `bign-with-hbelt` должны использоваться параметры `bign-curve256v1`, в алгоритмах `bign-with-bash384` — параметры `bign-curve384v1`, в алгоритмах `bign-with-bash512` — параметры `bign-curve512v1`.

Идентификаторы алгоритмов ЭЦП добавляются в перечисление `SignatureScheme` (см. 6.3.4):

```
SignatureScheme += {
    bign_with_hbelt(0xFE01),
    bign_with_bash384(0xFE02),
    bign_with_bash512(0xFE03)
};
```

Идентификаторы используются в расширениях `signature_algorithms` и `signature_algorithms_cert`.

Библиография

- [1] ISO/IEC 9899:2011, Information technology — Programming languages — C
(Информационные технологии. Языки программирования. Си)
- [2] Eisler M., ed. XDR: External Data Representation Standard. Request for Comments: 4506, 2006
(XDR: стандарт представления внешних данных)
- [3] Rescorla E. The Transport Layer Security (TLS) Protocol Version 1.3. Request for Comments: 8446, 2018
(Протокол защиты транспортного уровня версии 1.3)
- [4] Krawczyk H. Cryptographic Extraction and Key Derivation: The HKDF Scheme. In: Rabin, T. (eds) Advances in Cryptology — CRYPTO 2010. Lecture Notes in Computer Science, 6223: 237–254. Springer, Berlin, Heidelberg, 2010
(Извлечение и построение криптографических ключей: схема HKDF)
- [5] Krawczyk H., Eronen P. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). Request for Comments: 5869, 2010
(Функция построения ключей по схеме «извлечь-и-развернуть» на основе HMAC (HKDF))
- [6] Wouters P., Tschofenig H., Gilmore J., Weiler S., Kivinen T. Using Raw Public Keys in Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS). Request for Comments: 7250, 2014
(Использование самостоятельных открытых ключей в TLS и DTLS)
- [7] Eastlake 3rd D. Transport Layer Security (TLS) Extensions: Extension Definitions. Request for Comments: 6066, 2011
(Расширения TLS: определения расширений)
- [8] McGrew D., Rescorla E. Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP). Request for Comments: 5746, 2010
(Расширение DTLS для формирования ключей протокола SRTP)
- [9] Seggelmann R., Tuexen M., Williams M. Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS) Heartbeat Extension. Request for Comments: 6520, 2012
(Расширение Heartbeat протоколов TLS и DTLS)
- [10] Friedl S., Popov A., Langley A., Stephan E. Transport Layer Security (TLS) Application-Layer Protocol Negotiation Extension. Request for Comments: 7301, 2014
(Расширение TLS для согласования прикладного протокола)
- [11] Laurie B., Langley A., Casper E. Certificate Transparency. Request for Comments: 6962, 2013 (Обеспечение прозрачности сертификатов)
- [12] Langley A. A Transport Layer Security (TLS) ClientHello Padding Extension. Request for Comments: 7685, 2015 (Расширение TLS для увеличения размера ClientHello)

- [13] Moeller B., Langley A. TLS Fallback Signaling Cipher Suite Value (SCSV) for Preventing Protocol Downgrade Attacks. Request for Comments: 7507, 2015
(Криптонабор SCSV протокола TLS для защиты от атак по понижению версии)
- [14] Rescorla E. Keying Material Exporters for Transport Layer Security (TLS). Request for Comments: 5705, 2010
(Экспортеры ключевого материала для TLS)
- [15] Bhargavan K., Delignat-Lavaud A., Pironti A., Langley A., Ray M. Transport Layer Security (TLS) Session Hash and Extended Master Secret Extension. Request for Comments: 7627, 2015
(Хэширование стенограмм TLS и расширение для изменения алгоритма вычисления мастер-ключа)
- [16] Bhargavan K., Delignat-Lavaud A., Fournet C., Pironti A., Strub P.-Y. Triple Handshakes and Cookie Cutters: Breaking and Fixing Authentication over TLS. 2014 IEEE Symposium on Security and Privacy, Berkeley, CA, USA, 2014, pp. 98–113
(Тройное рукопожатие и обрезка куки: взлом и исправление аутентификации на основе TLS)